一、进制转换
1、二进制数据
2、十进制转换成二进制
3、八进制数据
4、二进制数据转换八进制
5、十六进制数据
6、二进制转换成十六进制
7、任意进制转换成十进制：
二、原码、反码、补码
1、什么是原码
2、什么是反码
3、什么是补码
4、补码转换成十进制整数：
无符号解析：
有符号解析：
三、位运算符
1、A & B 按位与运算
2、A | B 按位与运算
3、A ^ B 按位异或运算
4、~A 按位求反，是单目运算符
5、<< >> 位移运算
四、程序的内存分段：
1、text 代码段
2、data 数据段
3、bss 静态数据段
4、heap 堆
5、stack 栈
6、注意
五、变量属性和分类
1、变量的属性
2、全局变量
3、局部变量
4、块变量
5、变量同名问题
6、全局变量的优点和缺点
优点：
缺点：
总结：
六、修饰变量的关键字
1、typedef
2、auto
3、const
4、static
限制作用域：
改存储位置：
延长生命周期：
5、volatile
6、register
7、extern
七、计算机的内存长什么样子：
八、指针
1、什么是指针
2、如何使用指针
定义指针变量：
给指针变量赋值：

指针变量解引用：
3、为什么要使用指针：
1、函数之间共享变量：
2、使用指针变量可以节约函数的传参效率：
3、使用堆内存时必须与指针变量配合：
4、使用指针要注意的问题
1、空指针
2、如何避免空指针产生的段错误
3、野指针
4、对野指针解引用的后果
5、如何避免野指针产生的错误
6、如何不制造野指针
7、野指针比空指针的危害更大
九、指针的进步值与运算
1、指针的进步值
2、指针的运算
3、注意
十、数组名与指针
1、数组名的本质是什么
2、数组名与指针的相同点
3、数组名与指针的不同点
十一、通用指针
1、什么是通用指针
2、为什么需要使用通用指针
3、常用通用操作的函数
3、使用通用指针要注意的问题
十二、const与指针
1、保护指针变量指向的内存不被修改
2、保护指针变量不被修改
3、既保护指针也保护内存
十三、二级指针
1、什么是二级指针
2、定义二级指针
3、二级指针的赋值
3、二级指针解引用
4、二级指针的用处
十四、指针数组与数组指针
1、什么是指针数组
2、定义指针数组
3、指针数组的用处
4、什么是数组指针
5、定义数组指针
6、数组指针的用处
7、数组名、指针、数组指针
十五、函数指针
1、函数名是什么
2、什么是函数指针
3、定义函数指针：
4、函数指针的用处
十六、字符
1、什么是字符
2、重要字符的ASCII码值
3、字符的输出
3、字符的输入
4、输入字符时要注意的问题
5、判断字符类型的函数：
十七、输入、输出缓冲区
1、输出缓冲区

2、数据从输出缓冲区到达文件的条件有
3、输入缓冲区
4、清理输入缓冲区的方法
十八、字符串
1、什么是串型结构
2、什么是字符串
3、字符串字面值
4、字符串的输出
5、字符串的输入
6、操作字符串的常用函数
十九、电子通讯录作业
1、联系人信息：
2、功能
3、参考代码
二十、C语言的堆内存管理
1、malloc函数
2、使用malloc函数要注意的问题
3、calloc函数
4、使用calloc函数要注意的问题
5、free函数
6、使用free函数要注意的问题
7、realloc函数
8、使用realloc函数要注意的问题
9、总结
二一、堆内存越界
1、malloc管理堆内存的机制
2、为什么是135160会出现段错误
3、这种分配机制的优点
4、使用堆内存越界的后果：
二二、内存碎片
1、什么是内存碎片
2、内存碎片产生的原因
3、如何减少内存碎片
二三、内存泄漏
1、什么是内存泄漏
2、产生内存泄漏的原因
3、如何减少内存泄漏
4、检查内存泄漏的工具
5、内存泄漏和内存碎片的危害
二四、结构
1、什么是结构
2、如何设计结构
3、如何使用结构
4、初始化结构变量
5、访问结构成员
6、使用typedef重定义简短的类型名
二五、结构的总字节数
1、为什么要学会计算结构的总字节数
2、内存对齐
3、内存补齐
4、long类型的字节数
二六、联合
1、联合与结构的区别
2、联合的总字节数
3、使用联合的意义
二七、大端系统和小端系统
1、大端系统
2、小端系统

二八、枚举
1、设计枚举
2、定义枚举变量
3、枚举值
二九、预处理与预处理指令
1、文件包含指令
方法1：#include <file_name.h>
方法2：#include "file_name.h"
2、宏替换指令
1、定义宏名
2、使用宏名
3、宏常量
3、宏表达式
4、定义宏常量和宏表达式要注意的问题
5、枚举常量与宏常量的区别
6、预定义的宏
编译器预定义的宏：
标准库预定义的宏：
三十、宏函数
1、什么是宏函数
2、定义宏函数要注意的问题
3、宏的二义性
3、调用宏函数要注意的问题
4、普通函数与宏函数的优缺点?
1、宏函数的优点
2、宏函数的缺点
3、普通函数的优点
4、函数的缺点
5、什么样的代码适合封装成宏函数
三一、条件编译
1、负责条件编译的预处理指令
2、头文件卫士
3、注释代码
4、版本、环境判断
5、DEBUG宏
三二、不常用的预处理指令
宏函数的变长参数：
在编译时定义宏：
三三、多文件编程
1、一般的拆分方案
2、头文件中可以写什么
3、头文件包含可能出现的问题
互相包含:
递归包含：
解决方法：
4、多文件编译过程
三四、文件分类
1、二进制文件
2、文本文件
3、二进制文件与文本文件的区别
二进制文件的内容：
文本文件的内容：
三五、文件相关操作
1、打开与关闭的函数
2、文件的打开模式
3、Windwos系统下以文本方式打开文件时
4、文本文件的读写
5、二进制文件的读写

6、文件位置指针：
7、文件操作时的局限性：
向文件中插入数据：
从文件中删除数据：
8、文件管理

一、进制转换

日常生活中使用的是十进制数据，由于计算机的物理特性导致它只能识别二进制数据，我们在编程时使用的10进制数据，会被计算机转换成二进制数据再运算，为了理解编程时的一些奇怪现象，我们需要掌握二进制数据、八进制数据、十六进制数据。

1、二进制数据

由0~1两个数字组成，逢2进1，由于计算机的运算器只能识别高低两种电流，所以它在运算时只能识别二进制数据。

2、十进制转换成二进制

假如把x转换成二进制，x/2记录下余数，然后对商继续除以2，重复这个过程，直到商为0结束，然后把记录的余数倒序汇总，就得到了x的二进制。

```
例如：117转换成二进制
    117 / 2 余数1
    58 / 2 余数0
    29 / 2 余数1
    14 / 2 余数0
    7 / 2 余数1
    3 / 2 余数1
    1 / 2 余数1
    0
    117的二进制就是：1110101
```

练习：实现一个函数，能够打印十进制数据的二进制。

```c
#include <stdio.h>

void print_binary(int num)
{
    char bits[32],cnt = 0;
    while(num)
    {
        bits[cnt++] = num % 2;
        num /= 2;
    }

    for(int i=cnt-1; i>=0; i--)
    {
        printf("%hhd",bits[i]);
    }
}

int main()
{
    int num;
```

```
    printf("请输入一个整数：");
    scanf("%d",&num);
    print_binary(num);
}
```

3、八进制数据

由0~7八个数字组成，逢8进1，早期使用它记录二进制数据，现在基本不再使用，文件的权限还依然使用8进制数据表示，所以还无法退出历史。

4、二进制数据转换八进制

三位二进制对应一位八进制。

```
    000 0
    001 1
    010 2
    011 3
    100 4
    101 5
    110 6
    111 7
```

注意：在C代码中，以0开头的是八进制数据，以%o输出的也八进制数据。

5、十六进制数据

由0~9和a~f十六个字符组成，随着计算机的发展CPU的位数越来越多，输出的二进制也越来越长，随后科学家又发明出十六进制用于记用二进制数据。

6、二进制转换成十六进制

四位二进制对应一位十六进制，超过9的用字母ABCDEF表示（不区分大小写）。

```
    1000 8
    1001 9
    1010 a
    1011 b
    1100 c
    1101 d
    1110 e
    1111 f
```

注意：在C代码中，以0x开头的是十六进制数据，以%x，%p输出的是十六进制数据。

7、任意进制转换成十进制：

每一个非0位带该公式求和 v*b^(n-1) 就得到了十进制。
v 值
b 进制
n 位数

二、原码、反码、补码

1、什么是原码

整数的绝对值转换成二进制，叫原码。

2、什么是反码

原码按位求反，叫反码。

3、什么是补码

正数的原码就是补码，负数的反码+1是补码。

> 十进制的数据是以补码形式存储在计算机中的，因为计算机的CPU中只有加法器，也就是只能运算加法，其它运算都是使用加法模拟的。
>
> 为了能计算出a-b也就是a + -b 所以需要使用特殊格式存储负数。

```
24 - 15
24 + -15
00001111 -15的原码
11110000 -15的反码
11110001 -15的补码
00011000 24的补码
00001001 结果是9
```

4、补码转换成十进制整数：

补码的两种解析方式：

无符号解析：

由于无符号数据全部是正数，所以补码就是原码，直接转换成十进制即可。

有符号解析：

根据补码的最高位判断它是整数的补码还是负数的补码。

最高位是1：它必然是负数的补码。

1、补码-1得到反码

2、反码按位求反，得到原码

3、原码转换成十进制数据，并在结果的前面加上负号。

最高位是0：它必然是正数的补码，直接转换成十进制即可。

注意：当遇到补码转换成十进制数据的笔试题时，必须有是否是有符号信息。

```
// 假如以下变量的补码是10110011，代码会输出什么，补码的符号信息已经包含在代码中了
char num;
printf("%hhd",num);

// 把以下补码转换成十进制数据，条件不够充分
11001100
```

注意：一个整型变量的取值范围是环型状的，当它是最大值时再加1就会变成最小值，当它是最小值时减1就是变成最大值。

```
以char类型数据为例：
-128
10000000 补码
01111111 -1后的补码，转换成十进制的结果是127

127
01111111 补码
10000000 加1后的补码，转换成十进制的结果是-128

char特殊的补码：
10000000 最小值(最高位是1，其余全是0)
01111111 最大值(最高位是0，其余全是1)
11111111 -1(所有二进制位都是)
```

```
//以下循环执行多少次：
char cnt = 7;
while(cnt)
{
   cnt -= 3;
}
7     45
-128  1
125   42
-1    42
-127  1
126   42
0     173
```

作业1、实现一个函数，把一个正整数转换成N进制并显示 2<= N <= 36。

```
#include <stdio.h>

void print_num_N(unsigned int num,int N)
{
    char bits[32] , cnt = 0;
    while(num)
    {
        bits[cnt++] = num % N;
        num /= N;
    }

    for(int i=cnt-1; i>=0; i--)
    {
        if(bits[i] >= 10)
            printf("%c",bits[i]-10+'A');
        else
            printf("%hhd",bits[i]);
    }
    printf("\n");
}

int main()
{
    print_num_N(255,32);
    print_num_N(127,2);
}
```

作业2、实现一个函数，显示一个整数的补码(num可能是整数也可能是负数)。

```c
#include <stdlib.h>

void print_bits(int num)
{
    // 计算出原码
    char bits[32] = {}, cnt = 0;
    int tmp = num;
    while(num)
    {
        bits[cnt++] = abs(num % 2);
        num /= 2;
    }
    if(tmp < 0)
    {
        // 计算出反码
        char carry = 1;
        for(int i=0; i<32; i++)
        {
            bits[i] = !bits[i];
            bits[i] += carry;
            carry = bits[i] / 2;
            bits[i] %= 2;
        }
    }


    for(int i=31; i>=0; i--)
    {
        printf("%hhd",bits[i]);
    }
}

int main()
{
    print_bits(-127);
}
```

三、位运算符

位运算符是针对数据的补码进行运算。

1、A & B 按位与运算

```
0 & 0 结果是0
0 & 1 结果是0
1 & 0 结果是0
1 & 1 结果是1
```

2、A | B 按位与运算

```
0 | 0 结果是0
0 | 1 结果是1
1 | 0 结果是1
1 | 1 结果是1
```

3、A ^ B 按位异或运算

```
0 ^ 0 结果是0
0 ^ 1 结果是1
1 ^ 0 结果是1
1 ^ 1 结果是0
```

4、~A 按位求反，是单目运算符

```
~0 结果是1
~1 结果是0
```

5、<< >> 位移运算

```
x << n 把x的补码前n位丢掉，末尾补上n个0，按位左移。
    10101100 << 3 01100000
x >> n 把x的补码后n位丢掉，前面n位，如果x是正数则补0，如果是负数则补1。
    char num = -3;
    printf("%hhd\n",num >> 2 +1);
```

四、程序的内存分段：

当执行程序的运行命令后，操作系统会给程序分配它所需要的内存，并划分成以下内存段供程序使用：

1、text 代码段

C代码被翻译成二进制指令后存储在可执行文件中，当可执行文件被操作系统执行时，它会把里面的二进制指令加载到这个内存段，它里面的内容决定了程序如何执行，为了避免程序被破坏、修改，所以它的权限是只读。

该内存段分为两个部分：

r-x：二进制指令　r--：常量数据

注意：该内存段的内容如果被强制修改会产生段错误（非法使用内存）。

2、data 数据段

存储的是初始化过的全局变量

存储在内存段的变量，被const修饰后，就会改存储到text内存段，变成真正的常量。

3、bss 静态数据段

存储的是未初始化的全局变量

操作系统把程序被加载到内存后，会把该内存段进行初始化，也就是所有字节赋值为零，所以全局变量的默认值不是随机，而是零。

4、heap 堆

该内存段由程序员手动调用内存管理函数(malloc/free)，进行分配、释放，它的分配释放受程序员的控制，适合存储一些需要长期使用的数据。

它的大小不受限制，理论上能达到物理的上限，所以适合存储大量的数据。

该内存段无法取名字，也就是无法与标识符建立联系，必须与指针配合使用。

5、stack 栈

存储的是局部变量、块变量

该内存段会随着程序的执行自动的分配(定义局部变量、块变量)、释放(函数执行完毕自动释放局部变量、块变量)，虽然使用比较方便，但它的释放不受程序控制，长期使用的数据不能存储在栈内存中。

该内存的大小有限，在终端执行： ulimit -s 可以查看当前系统栈内存的使用上限，我们使用虚拟机ubuntu的栈内存使用上限是8192kb，一旦超过这个限制就会产生段错误。可以使用ulimit -s 命令设置栈内存的使用上限。

6、注意

1、当程序完成编译 text、data、bss 三个内存段的大小就确定，在程序运行期间大小不会有任何变化，可以使用size命令查看程序的这三个内存段的大小。

2、heap、stack两个内存段，会随着程序的执行动态变化。

3、当程序运行时，/proc/进程号/maps 文件里记录程序执行过程中内存的使用情况，程序运行结束这个文件就消失了。

五、变量属性和分类

1、变量的属性

- 作用域：变量的使用范围。
- 存储位置：变量使用那个内存段存储数据。
- 生命周期：变量从定义、分内存到内存销毁的时间段。

2、全局变量

定义在函数外的变量叫全局变量。

- 作用域：任何位置都可以使用。
- 存储位置：初始化的全局变量使用的是data内存段，未初始化的全局变量使用的是bss内存段。
- 生命周期：从程序开始执行，到程序执行结束。

3、局部变量

定义在函数内的变量叫局部变量。

- 作用域：只能在它所在的函数内使用。
- 存储位置：使用的是stack内存段。

- 生命周期：当它所在的函数被调用后，执行到局部变量的定义语句时局部变量就会被创建（操作系统会给局部变量的变量名分配一块stack内存），当函数执行结束后，局部变量就被销毁了。

4、块变量

定义在if、for、while、do while语句块内的变量叫局部变量，就是特殊的局部变量。

- 作用域：只能在它所在的语句块内使用。
- 存储位置：使用的是stack内存段。
- 生命周期：当它所在的函数被调用后，执行到块变量的定义语句时块变量就会被创建（操作系统会给块变量的变量名分配一块stack内存），当出了它所在的大括号，块变量就被销毁了。

5、变量同名问题

同一个作用域下的变量不能重名，否则编译就会报错。

全局变量、局部变量、块变量的作用域不同，所以可以同名，局部变量会屏蔽同名的全局变量，块变量会屏蔽同名的全局变量、局部变量。

6、全局变量的优点和缺点

优点：

使用方便，避免了函数之间传参产生的消耗，提高程序的运行速度。

缺点：

程序运行期间全局变量所占用的内存不会被销毁，可能会产生内存浪费。

命名冲突的可能性比较大，可能会与其它文件的全局变量、函数、结构、联合、枚举、宏命名冲突。

总结：

全局变量尽量少用，或者不用。

六、修饰变量的关键字

<类型限定符> 数据类型 变量名;

1、typedef

变量被typedef修饰后，就会变成定义它的数据类型，之后就可以使用这种新的数据类型定义变量、数组了，该功能是为了给复杂的数据类型重新定义一个简短的类型名。

由于无符号整型使用比较麻烦，所以标准库中为我们定义一些简短的无符号整型的类型名，就使用typedef定义的，实现在stdint.h头文件里。

```
typedef signed char     int8_t;
typedef short int       int16_t;
typedef int             int32_t;
typedef long long int   int64_t;

typedef unsigned char       uint8_t;
typedef unsigned short int  uint16_t;
typedef unsigned int        uint32_t;
typedef unsigned long long int  uint64_t;
```

注意：在之后的学习过程中，如果遇到一些xxx_t的数据类型，都使用typedef重定义。

2、auto

早期的C语言用它来修饰自动分配、释放内存的变量，也就是局部变量和块变量，但由于代码使用的变量绝大多数都是局部变量和块变量，所以就约定，该关键字不加就代码加，所以该关键字已经没有实用价值了。

在C++11的语法标准中，auto有了新的功能，就是定义自动类型的变量，编译器会根据变量的初始值，自动设置变量的数据类型。

```
int main()
{
    auto num = 3.14;
    printf("%d",sizeof(num));
}
// 编译指令：g++ xxx.c -std=c++11
```

注意：虽然auto关键字，已经不再使用，但基本功能还保留着，所以它不能修饰全局变量。

3、const

const的意思是常量，但实际它只是为变量提供一层保护，被它修饰的变量不能显式，但可以隐式修改，也就被它修饰后并不能变成真正的常量。

```
#include <stdio.h>

int main()
{
    const int num = 1234;

    int* p = (int*)&num;
    *p = 6666;

    printf("%d\n",num);

}
// 执行结果是：6666
```

注意：存储在data内存段的变量，被const修饰后就会变成真正的常量，存储位置被修改为text，其实是修改了data段和text段的分界线。

```
#include <stdio.h>

const int num = 1234;

int main()
{

    int* p = (int*)&num;
    *p = 6666;

    printf("%d\n",num);
}
// 执行后会出现段错误
```

4、static

static既可以修饰变量，也可以修饰函数，主要有三大功能：

限制作用域：

1、全局变量、函数的作用域是整个程序都可以使用，被static修饰后，就只能在它所在的.c文件内使用。

2、该功能可以避免全局变量、函数的命令冲突，也能防止全局变量、函数被外部修改、调用，提高代码的安全性。

3、普通全局变量、函数也叫外部变量、外部函数，被static修饰后就叫做内部变量、内部函数、静态全局变量。

改存储位置：

1、局部变量、块变量被static修饰后，存储位置就由stack改data、bss，称呼为静态局部变量、静态块变量。

2、静态局部变量、静态块变量的默认值不再是随机的，而是零。

延长生命周期：

由于静态局部变量、静态块变量的存储位置由stack(动态分配、释放)改为data、bss，所以静态局部变量、静态块变量不会随着函数的执行结束而销毁，而是和全局变量的生成周期一样。

```c
#include <stdio.h>

void func(int i)
{
    static int num = 1;
    num *= i;
    printf("%d\n",num);
}

int main()
{
    for(int i=1; i<10; i++)
    {
        func(i);
    }
}
```

注意：static修饰局部变量、块变量，会改变它们的存储、延长生命周期，但并不会改变它们的作用域。

5、volatile

```c
int num = 123;
...
if(num == num)
{

}
// 默认情况下，比较结果永远为真


```

```
volatile int num = 123;
...
if(num == num)
{

}
// num被volatile修饰后，比较结果可能不为真
```

在程序中使用到num变量时，系统会从内存中读取该num的值交给CPU运算，如果之后num变量的值没有发生明显变化，再次使用变量时系统会直接使用上次读取的旧值，而不会再从内存中读取。这编译器会变量读值过程的优化。

volatile 关键字就告诉编译器不要优化变量的读值过程，每使用该变量时，都重新从内存中读取它的值。

什么情况下需要使用volatile关键字：

变量被共享访问，且有多个执行者可以修改它的值，这种情况下变量就应用被volatile修饰。

情况1：多线程编程处理复杂问题时。

情况2：裸机编程、驱动编程时，软硬件共用的寄存器。

6、register

计算机的存储介质读写速度排序：机械硬盘->固态硬盘->内存条->高级缓存->CPU寄存器

register关键字的作用是申请把变量的存储介质由内存条改为CPU寄存器，一旦申请成功，变量的读写速度、运算速度会大大提高。

```
#include <stdio.h>

int main()
{
    int index = 0;
    while(index < 0x7fffffff)
        index++;
}
/*
real    0m0.463s
user    0m0.463s
sys 0m0.001s
*/

#include <stdio.h>

int main()
{
    register int index = 0;
    while(index < 0x7fffffff)
        index++;
}
/*
real    0m0.463s
user    0m0.463s
sys     0m0.001s
*/
```

注意：CPU中的寄存器数量有限，申请不一定成功，只有需要长期大量运算的变量才适合用register关键字修饰。

7、extern

当使用其它.c文件中的全局变量时，需要像声明函数一样，对其它.c文件全局变量进行声明。

```
extern 类型 变量名;
```

注意：声明变量只能解决编译时的问题，如果目标文件最终链接时，变量没有定义，依然会报错。

七、计算机的内存长什么样子：

1、计算机的内存就像是一叠非常厚的"**便签**"，一张便签就相当于一个字节的内存，一个字节有8个二进制位。

2、每一张"**便签**"的都有自然排序形成的一个编号，计算机根据便签的编号访问、使用"**便签**"。

3、CPU会有若干个金手指，每根金手指能感知高低两种电流，低电流当作二进制的0，高电流当作二进制的1，我们所说的32位的CPU，指的是CPU有32个金手指用于感知便签的编号：

```
便签的最小编号 00000000000000000000000000000000 = 0
便签的最大编程 11111111111111111111111111111111 = 4294967295
所以32位的CPU最多能使用 4294967296byte -> 4194304kb -> 4096mb -> 4gb
```

4、便签的编号也就是内存的地址，是一种无符号整数。

八、指针

1、什么是指针

1、指针（pointer）是一种数据类型，使用它可以定义指针变量，简称指针。

2、指针变量中存储的是内存的地址，是一种无符号的整数。

3、通过指针变量中记录的内存地址，我们可以读取内存中所存储的数据，也也可以向，内存中写入数据。

4、一般使用%p以十六进制格式显示内存地址。

2、如何使用指针

定义指针变量：

> 类型* 指针变量名;

1、指针变量中只记录了内存中某字节的地址编号，我们把它当作一个内存块的首地址，当使用指针变量访问内存时具体访问多少个字节，由指针变量的类型决定。

```
char* p;    // 能访问1字节
short* p;   // 能访问2字节
int* p;     // 能访问4字节
```

2、普通变量与指针变量的用法不同，为了避免混用，所以指针变量一般以p结尾。

3、指针变量不能连续定义，一个*只能定义出一个指针变量。

```c
int num1,num2,num3;
int* p1,p2,p3;      // p1是指针变量，p2、p3是普通的int类型变量
int *p1,*p2,*p3;    // p1、p2、p3都是指针变量

typedef int* intp;
intp p1,p2,p3;      // p1、p2、p3都是指针变量
```

4、指针变量与普通变量一样，默认值是随机的（野指针），为了安全尽量给指针变量初始化，如果不知道该赋什么值，可以先初始化为NULL（空指针）。

给指针变量赋值：

> 指针变量 = 内存地址。

所谓的给指针变量赋值，就是给指针变量存储一个内存地址，如果该内存地址是非法的，当使用指针变量访问内存是就会出现段错误。

```c
int* p = malloc(4); // 把堆内存的地址赋值给指针变量
int* p = &num;      // &计算出变量的内存地址(单目运算符)
注意：num变量的类型必须与p的类型相同
```

指针变量解引用：

> *指针变量

指针变量赋值就是引用一块内存，解引用就是根据指针变量存储的内存地址，去访问内存，具体访问多少个字节由指针变量的类型决定。

如果指针变量中存储的是非法内存地址，该动作会出现段错误，要从指针变量赋值的步骤去解决。

```c
int num = 1234;
int* p = &num;

*p <=> num;
printf("%d",*p);
*p = 2345;
printf("%d\n",num);
```

```c
int main()
{
    const int num = 1234;
    int* p = (int*)&num;
    *p = 6666;
    printf("%d\n",num);
}
```

3、为什么要使用指针：

1、函数之间共享变量：

函数之间的命名空间是互相独立，并且以赋值方式传参的，所以传参无法解决共享变量的问题

全局变量虽然可以在函数之间共享，但过多使用全局变量可以会造成命名冲突和内存浪费。

当函数一个需要返回两个以上的参数时，就需要共享变量了。

2、使用指针变量可以节约函数的传参效率：

函数之以赋值方式传参的，也就是内存的拷贝，把一个变量的内存内容拷贝给别一个变量，当变量的字节数比较大时（大于4字节），传参效率就很低，而传递变量的地址，只需要拷贝4字节内存。

```c
#include <stdio.h>


void func(long double f)
{

}
int main()
{
    long double f = 3.14;
    for(int i=0; i<1000000000; i++)
    {
        func(f);
        f++;
    }
}
/*
real    0m5.527s
user    0m5.523s
sys     0m0.004s
*/

void func(long double f)
{

}
int main()
{
    long double f = 3.14;
    for(int i=0; i<1000000000; i++)
    {
        func(f);
        f++;
    }
}
/*
real    0m2.553s
user    0m2.553s
sys     0m0.000s
*/
```

3、使用堆内存时必须与指针变量配合：

堆内存无法取名字，标准库、操作系统提供内存分配接口的返回值都内存地址，所以必须使用变量配合才能使用堆内存。

```c
void *malloc(size_t size);
void *calloc(size_t nmemb, size_t size);
void *realloc(void *ptr, size_t size);
```

注意：由于使用指针变量具有一定的危险，所以除了以情况，不要轻易使用指针。

4、使用指针要注意的问题

1、空指针

指针变量中如果存储的是NULL，那么它就是空指针，因此操作系统规定程序不能访问该内存，只要访问就会产生段错误，同时也是返回值是指针类型的函数执行错误的标志。

2、如何避免空指针产生的段错误

对来历不明的指针进行解引用前，要先判断是否是空指针。

1、当指针变量接收了函数的返回值，判断是否是空指针，既能避免访问空指针产生的段错误，也能知道该函数执行是否失败、出错。

2、如果设计的函数参数是指针变量，那么调用者传递实参就可能是空指针，对指针变量解引用前要先判断。

```
if(NULL == p) // 正确写法
{

}

if(p == NULL) // 错误写法，当少写一个等号时，就变成了给p赋值为空指针
{

}

if(!p)  //大多数系统的NULL是0，但有少数系统的NULL是1
{

}
```

3、野指针

指针变量中存储的地址，无法确定是否是合法的内存地址，这种指针变量被称为野指针。

4、对野指针解引用的后果

1、一切正常，指针变量恰好存储的是空闲的内存地址，概率不高。

2、段错误，存储的是非法的内存地址。

3、脏数据，存储的是其它变量的内存地址。

5、如何避免野指针产生的错误

野指针无法判断出来，但所有的野指针都是人为制造出来的，所以要想避免野指针产生的错误，只能不制造野指针。

6、如何不制造野指针

1、定义指针变量时一定要初始化，要么赋值一个合法的内存地址，要么初始化NULL。

2、不返回局部变量、块变量的地址，当函数执行完毕后，局部变量和块变量就被销毁。

3、与堆内存配合的指针变量，当堆内存被释放、销毁，该指针变量要及时的赋值为NULL。

7、野指针比空指针的危害更大

野指针产生的错误具有隐藏性、潜伏性、随机性，所以野指针比空指针危害更大。

练习1、实现一个函数，用于交换两个int变量的值。

```c
#include <stdio.h>

void swap(int* p1,int* p2)
{
    if(NULL == p1 || NULL == p2)
        return;

    int tmp = *p1;
    *p1 = *p2;
    *p2 = tmp;
}

int main()
{
    int num1 = 3, num2 = 7;
    printf("%d %d\n",num1,num2);
    swap(&num1,&num2);

    printf("%d %d\n",num1,num2);
}
```

练习2、实现一个函数，用于计算两整数最大公约数和最小公倍数。

```c
#include <stdio.h>

// 最大公约数据使用返回值，最小公倍数存储在调用者提供的指针所指向的内存
int max_min(int num1,int num2,int* result)
{
    if(NULL == result)
        return -1;
    /*
    int max = 1;
    for(int i=2; i<=num1; i++)
    {
        if(0 == num1%i && 0 == num2 % i)
            max = i;
    }

    *result = num1*num2/max;
    return max;
    */

    for(int i=num1*num2; i>=num1; i--)
    {
        if(0 == i%num1 && 0 == i%num2)
            *result = i;
    }

    return num1 * num2 / *result;
}

```

```c
int main()
{
    int min;


    int max = max_min(4,6,&min);

    printf("%d %d\n",max,min);
}
```

九、指针的进步值与运算

指针变量里面存储的是整数，代表着内存的编号（每个整数都对应着一字节的内存）。

```c
#include <stdio.h>


void func(int num)
{
    *(int*)num = 6666;
}

int main()
{
    int num = 1234;
    func((int)&num);
    printf("%d\n",num);
}
```

1、指针的进步值

指针变量中存储的其实是一个内存块的首地址，内存块的具体大小由指针变量的类型决定，当使用指针变量解引用访问内存时，实际访问的内存字节数叫做指针变量的进步值，也就是指针变量+1后的内存地址的变化。

```c
#include <stdio.h>


int main()
{
    char* p1 = NULL;
    short* p2 = NULL;
    int* p3 = NULL;
    long long* p4 = NULL;
    long double* p5 = NULL;
    printf("%p %p %p %p %p\n",p1,p2,p3,p4,p5);
    printf("%p %p %p %p %p\n",p1+1,p2+1,p3+1,p4+1,p5+1);
}
```

2、指针的运算

指针变量存储就是是整数，理论上整数能使用的运算符，指针变量都可以使用，但只有以下运算才有意义：

```
指针+n = 指针所代表的整数+进步值*n
指针-n = 指针所代表的整数-进步值*n
指针1-指针2 = (指针1所代表的整数-指针2所代表的整数)/进步值
```

指针加减整数，就相当于以指针变量的进步值为单位前后移动，指针-指针可以计算出两个指针变量之间相隔多个元素。

```c
#include <stdio.h>

int main()
{
    int arr[10];
    int* p1 = &arr[0];
    int* p2 = &arr[9];
    printf("%p %p\n",p2,p1);
    printf("%d\n",p2-p1);

}
```

3、注意

指针-指针运算，它们的类型必须相同，否则编译器会报错。

十、数组名与指针

1、数组名的本质是什么

1、数组名就是数组内存块的首地址，它是个常量地址，所以它作函数的参数时，才能蜕变成指针变量。

```c
#include <stdio.h>

void func(int arr[],size_t len) // void func(int* arr,size_t len)
{
    int arr2[10] = {11,12,13,14,15,16,17,18,19,20};
    arr = arr2; // 可以成功赋值，所以arr是变量

    for(int i=0; i<10; i++)
    {
        printf("%d\n",arr[i]);
    }
}

int main()
{
    int arr[10] = {1,2,3,4,5,6,7,8,9,10};
    printf("%p %p\n",arr,&arr[0]);
    func(arr,sizeof(arr)/sizeof(arr[0]));
}
```

2、指针变量可以使用[]解引用，数组名也可以*遍历，它们是等价的。

```c
#include <stdio.h>

int main()
{
    int arr[10] = {1,2,3,4,5,6,7,8,9,10};
    for(int i=0; i<10; i++)
    {
        // 指针[i] <=> *(指针+i)
        printf("%d %d\n",arr[i],*(arr+i));

    }
}
```

3、数组名就是指针，并且是常量指针，但它作函数的参数是就会变成指针变量。

如果定义int arr[n]数组，数组名arr 就是 int*类型的地址。

2、数组名与指针的相同点

1、它们都是地址

2、它们都使用[],*去访问一块连续的内存

3、数组名与指针的不同点

1、数组名是常量，而指针是变量

2、指针变量有它自己的存储空间，而数组名就是地址，它没有存储地址的内存。

3、指针变量与它的目标内存是指向关系，而数组名与它的目标内存是映射关系。

```c
#include <stdio.h>

int main()
{
    int arr[10];
    int* p = arr;
    printf("%p %p\n",p,arr);

    printf("%p %p\n",&p,&arr);
}
```

十一、通用指针

1、什么是通用指针

在C语言中**void**类型的指针可以与其它类型的指针自动进行类型转换，所以它被称为通用指针。

```c
void* p1;
int* p2 = p1;
void* p3 = p2;
```

2、为什么需要使用通用指针

我们一些通用的操作函数，它们的参数可能是任意类型的指针，但编译器规定不同类型的指针不能进行赋值，为了兼容各种类型的指针，C语言中设计了void类型的指针，它能与任意类型的指针互相转换，它能解决不同类型的指针参数的兼容性问题。

3、常用通用操作的函数

```
void bzero(void *s, size_t n);
功能：把内存块s的n个字节，赋值为0。

void *memset(void *s, int c, size_t n);
功能：把内存块s的n个字节，赋值为c(0~255)

void *memcpy(void *dest, const void *src, size_t n);
功能：从src内存块拷贝n个字节的内容到dest内存块

int memcmp(const void *s1, const void *s2, size_t n);
功能：比较s1和s2内存块的n个字节
    s1 > s2 返回1
    s1 < s2 返回-1
    s1 == s2 返回0
```

3、使用通用指针要注意的问题

1、**void**类型的指针变量的进步值是1。

2、**void**类型的指针变量不能解引用，必须转换成其它类型的指针才能解引用。

练习：交换任意类型的两个变量的值。

```
#include <stdio.h>
#include <string.h>

void swap_mem(void* p1,void* p2,size_t n)
{
    if(NULL == p1 || NULL == p2)
        return;

    char tmp[n];
    memcpy(tmp,p1,n);
    memcpy(p1,p2,n);
    memcpy(p2,tmp,n);
}

int main()
{
    short num1 = 3,num2 = 7;
    printf("%hd %hd\n",num1,num2);
    swap_mem(&num1,&num2,sizeof(num1));
    printf("%hd %hd\n",num1,num2);

}
```

十二、const与指针

1、保护指针变量指向的内存不被修改

```
int main()
{
    int num = 1234;
    const int * p1 = &num;
    int const * p2 = &num;
    *p1 = 6666; // 编译报错
    *p2 = 6666; // 编译报错
}
```

为了提高函数的传参效率，我们会选择传递变量的地址而不是变量的值，这样虽然提高了传参的效率，但变量也有了被修改了风险，可以使用const 类型* 指针变量 声明函数的参数格式，防止函数的实现者误修改变量的值。

由于指针的使用存在一定的风险，所以函数的参数只要是指针，并且函数没有修改指针所指向的内存的需求，我们就应该给指针变量加上const 类型* 指针变量。

2、保护指针变量不被修改

```
int main()
{
    int num = 1234;
    int * const p = &num;
    p = NULL; // 编译报错
}
```

当使用数组作函数的参数时，数组就蜕变成了指针变量，为了防止指针改变指向导致数组无法使用，我们应该使用 类型* const 指针变量 防止指针变量改变指向。

与堆内存配合的指针变量也应该从一而终，这样定义 类型* const 指针变量 防止指针变量改指向导致堆内存无法释放（防止产生内存泄漏）。

3、既保护指针也保护内存

```
const int * const p;
功能：既保存指针变量p不被修改，也保护指针变量指向的内存*p不被修改

int const * const p;
功能：同上
```

十三、二级指针

1、什么是二级指针

一级指针存储的是普通变量的内存地址，二级指针存储的是指针变量内存地址。

2、定义二级指针

```
int main()
{
    int* p;   // 一级指针
    int** pp; // 二级指针
}
```

注意：二级指针在使用方法上与一组指针不同，所以一般以pp结尾，让使用者从变量名上就能区别一级指针与二级指针。

3、二级指针的赋值

```
int main()
{
    int num = 1234;
    int* p = &num; // 给一级指针赋值
    int** pp = &p; // 给二级指针赋值
}
```

注意：给二级指针赋值的一级指针，它们的类型必须相同，否则编译时就会报错。

3、二级指针解引用

二级指针里面存储的是一级指针变量的地址，所以它可以解两级引用：

*二级指针 此时它等价于一级指针

**二级指针 此时它等价于 *一级指针

```
#include <stdio.h>

int main()
{
    int num = 1234;
    int num1 = 4567;
    int* p = &num;
    int** pp = &p;

    *pp = &num1; // p = &num1;
    printf("%d\n",*p);

    **pp = 123456789; // num1 = 123456789;
    printf("%d\n",num1);

}
```

4、二级指针的用处

只有一个情况适合使用二级指针，那就是跨函数共享一级指针变量。

十四、指针数组与数组指针

1、什么是指针数组

由指针变量构成的数组，也可以说它的身份是数组，成员是指针变量。

2、定义指针数组

类型* 数组名[n];

就相当于定义了n个指针变量。

3、指针数组的用处

1、构建不规则二维数组。

2、构建字符串数组。

```c
#include <stdio.h>

int main()
{
    int arr1[] = {6,1,2,3,4,5,6};
    int arr2[] = {3,1,2,3};
    int arr3[] = {5,1,2,3,4,5};
    int arr4[] = {4,1,2,3,4};

    int* arr[] = {arr1,arr2,arr3,arr4};

    for(int i=0; i<4; i++)
    {
        for(int j=1; j<=arr[i][0]; j++)
        {
            printf("%d ",arr[i][j]);
        }
        printf("\n");
    }
}
```

4、什么是数组指针

专门指向数组的指针变量，它的进步值是整个数组的字节数。

5、定义数组指针

类型 (*指针变量名) [n];

类型和n决定了 数组指针 指向的是什么样的数组。

```c
#include <stdio.h>

int main()
{
    int arr[5] = {1,2,3,4,5};
    int (*p)[5] = &arr;
    printf("%d\n",((int)(p+1))-((int)p));

    printf("%p %p %d\n",p,p+1,*((int*)((int)(p+1)-4)));
}
```

6、数组指针的用处

```c
#include <stdio.h>
// 二维数组就是一块连接的内存空间，只是逻辑把它划分成了5行4列
int main()
{
    int arr[5][4] = {
        {1,2,3,4},
        {5,6,7,8},
        {9,10,11,12},

        {13,14,15,16},
        {17,18,19,20}
    };
    int* p = (void*)arr;
    for(int i=0; i<20; i++)
    {
        printf("%d ",p[i]);
    }
}
```

```c
#include <stdio.h>
// 使用数组指针可以把一块连续的内存当作二维数组使用，特别是与堆内存配合效果更佳
int main()
{
    int arr[20] = {1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20};
    int (*p)[5] = (void*)arr;

    for(int j=0; j<4; j++)
    {
        for(int i=0; i<5; i++)
        {
            //printf("%d ",*(*(p+j)+i));
            printf("%d ",p[j][i]);
        }
        printf("\n");
    }
}
```

7、数组名、指针、数组指针

```
int arr[n];
arr 是 int*类型
*arr 是 int 类型
&arr[0] 也是 int*类型
&arr 是 int (*)[j]类型
arr[i] <=> *(arr+i)

int arr2[row][col];
arr2 是 int (*)[col]类型
*arr2 是 int* 类型
**arr2 是 int 类型
arr2[i][j] <=> *(*(arr2+i)+j)
&arr2[0] 是 int (*)[col]类型
&arr2[0][0] 是 int* 类型
```

十五、函数指针

1、函数名是什么

函数就是一段具有某项功能的代码，它会被编译器编译成二进制指令存储在text内存段，函数名就是它在text内存的首地址。

```c
#include <stdio.h>

void func(void)
{
    printf("我是函数func,我被调用了...\n");
}

int main()
{
    printf("%p\n",func);
    int num = 0x804843b;
    ((void(*)(void))num)();

}
```

2、什么是函数指针

专门存储函数地址的指针变量。

3、定义函数指针：

1、先确定指向的是函数的格式(函数声明)。

2、照抄函数声明。

3、用小括号包含函数名。

4、在函数名前加*

5、在函数名末尾加_fp，防止命名冲突。

6、用函数名给函数指针赋值后，函数指针就可以当作函数调用了。

```c
#include <stdio.h>

void func(void)
{
    printf("我是函数func,我被调用了...\n");
}

int main()
{
    void (*func_fp)(void) = func;
    func_fp();

}
```

4、函数指针的用处

函数指针可以让函数像数据一样在函数之间传递。

当我们实现一个数组的排序函数时，那么排序函数内部需要调用数组元素的比较函数，由于我们不知道待排序的数组是什么类型，也就无法自己实现数组元素的比较函数，那么我们可以在排序函数的参数列表中预留一个函数指针，当有人调我们的排序函数时，他就需要提供一个数组元素比较函数供我们调用，排序函数就可以为它的数组进行排序。

函数的这种调用模式就叫回调模式。

```c
void qsort(void *base,
           size_t nmemb,
           size_t size,
           int (*compar)(const void *, const void *));
功能：为数组进行排序
base：数组的首地址
nmemb：数组的长度
size：数组成员的字节数
compar：调用者需要提供的数组元素的比较函数
```

```c
#include <stdlib.h>

int intcmp(const void* p1,const void* p2)
{
    if(*(int*)p1 > *(int*)p2)
        return 1;

    if(*(int*)p1 < *(int*)p2)
        return -1;
    return 0;
}

int main()
{
    int arr[100];
    for(int i=0; i<100; i++)
    {
        arr[i] = rand() % 100;
    }

    qsort(arr,100,4,intcmp);
```

```

    for(int i=0; i<100; i++)
    {
        printf("%d ",arr[i]);
    }
}
```

练习1、实现一个通用的选择排序函数。

```
#include <stdio.h>
#include <assert.h>
#include <string.h>

void select_sort(void* base,
                 size_t len,
                 size_t bytes,
                 int (*comper)(const void*,const void*))
{
    assert(NULL!=base && NULL!=comper);

    char tmp[bytes];
    for(int i=0; i<len-1; i++)
    {
        int min = i;
        for(int j=i+1; j<len; j++)
        {
            if(0 < comper(base+min*bytes,base+j*bytes))
                min = j;
        }

        if(min != i)
        {
            memcpy(tmp,base+min*bytes,bytes);
            memcpy(base+min*bytes,base+i*bytes,bytes);
            memcpy(base+i*bytes,tmp,bytes);
        }
    }
}

int main()
{
    float arr[] = {9.1,6.6,3.2,1,6,5,7,4.4};
    int floatcmp(const void* p1,const void* p2)
    {
        if(*(float*)p1 > *(float*)p2)
            return 1;
        if(*(float*)p1 < *(float*)p2)
            return -1;
        return 0;
    }
    select_sort(arr,8,4,floatcmp);
    for(int i=0; i<8; i++)
    {
        printf("%g ",arr[i]);
    }

}
```

练习2、参考memcpy函数，实现自定义的内存拷贝函数，要考虑dest与src与交集的情况。

```c
#include <stdio.h>

void* my_memcpy(void* dest,const void* src,int n)
{
    if(NULL == dest || NULL == src || n < 1)
        return NULL;

    if(dest > src)
    {
        for(int i=n-1; i>=0; i--)
        {
            *(char*)(dest+i) = *(char*)(src+i);
        }
        return dest;
    }

    for(int i=0; i<n; i++)
    {
        *(char*)(dest+i) = *(char*)(src+i);
    }
    return dest;
}

int main()
{
    /*
    int src[] = {1,3,5,7,9};
    int len = sizeof(src)/sizeof(src[0]);

    int dest[len];
    my_memcpy(dest,src,sizeof(src));
    for(int i=0; i<len; i++)
    {
        printf("%d ",dest[i]);
    }
    */
    int arr[10] = {1,2,3,4,5,6,7,8,9,10};
    my_memcpy(arr+3,arr,20); // 1 2 3 1 2 3 4 5 9 10
    for(int i=0; i<10; i++)
    {
        printf("%d ",arr[i]);
    }

}
```

练习1、参考memcmp函数，实现自定义的内存比较函数。

```c
#include <stdio.h>
#include <stdint.h>
#include <assert.h>

int my_memcmp(const void* s1,const void* s2,int n)
{
    // 设置判断条件，当条件为假时，会让程序直接死掉，但会显示位置信息
    assert(NULL != s1 && NULL != s2 && n >= 0);
```

```c

    for(int i=0; i<n; i++)
    {
        if(*((uint8_t*)s1+i) > *((uint8_t*)s2+i))
            return 1;
        if(*((uint8_t*)s1+i) < *((uint8_t*)s2+i))
            return -1;
    }
    return 0;
}

int main()
{
    int arr1[] = {1,-2,3,4,5};
    int arr2[] = {1,2,4,3,5};
    printf("%d\n",my_memcmp(arr1,arr2,sizeof(arr1)));
}
```

十六、字符

1、什么是字符

字符就是符号或图案，但在计算机中以整数形式存在，当需要显示时，会根据ASCII表中的对应关系显示出相应的符号或图案。

在C语言中使用char类型的变量存储字符的ASCII码值，也就是使用整数进行模拟字符，标准的ASCII码表的范围是：0~127，共128个字符，其他的语种，使用-128~-1进行设计字符编码，比如中文的汉字，使用的是2~3字节存储一个汉字。

ASCII表

(American Standard Code for Information Interchange 美国标准信息交换代码)

高四位 / 低四位		ASCII控制字符												ASCII打印字符												
		0000						0001						0010		0011		0100		0101		0110		0111		
		0						1						2		3		4		5		6		7		
		十进制	字符	Ctrl	代码	转义字符	字符解释	十进制	字符	Ctrl	代码	转义字符	字符解释	十进制	字符	十进制	字符	十进制	字符	十进制	字符	十进制	字符	十进制	字符	Ctrl
0000	0	0		^@	NUL	\0	空字符	16	▶	^P	DLE		数据链路转义	32		48	0	64	@	80	P	96	`	112	p	
0001	1	1	☺	^A	SOH		标题开始	17	◀	^Q	DC1		设备控制 1	33	!	49	1	65	A	81	Q	97	a	113	q	
0010	2	2	☻	^B	STX		正文开始	18	↕	^R	DC2		设备控制 2	34	"	50	2	66	B	82	R	98	b	114	r	
0011	3	3	♥	^C	ETX		正文结束	19	‼	^S	DC3		设备控制 3	35	#	51	3	67	C	83	S	99	c	115	s	
0100	4	4	♦	^D	EOT		传输结束	20	¶	^T	DC4		设备控制 4	36	$	52	4	68	D	84	T	100	d	116	t	
0101	5	5	♣	^E	ENQ		查询	21	§	^U	NAK		否定应答	37	%	53	5	69	E	85	U	101	e	117	u	
0110	6	6	♠	^F	ACK		肯定应答	22	▬	^V	SYN		同步空闲	38	&	54	6	70	F	86	V	102	f	118	v	
0111	7	7	•	^G	BEL	\a	响铃	23	↨	^W	ETB		传输块结束	39	'	55	7	71	G	87	W	103	g	119	w	
1000	8	8	◘	^H	BS	\b	退格	24	↑	^X	CAN		取消	40	(	56	8	72	H	88	X	104	h	120	x	
1001	9	9	○	^I	HT	\t	横向制表	25	↓	^Y	EM		介质结束	41	)	57	9	73	I	89	Y	105	i	121	y	
1010	A	10	◙	^J	LF	\n	换行	26	→	^Z	SUB		替代	42	*	58	:	74	J	90	Z	106	j	122	z	
1011	B	11	♂	^K	VT	\v	纵向制表	27	←	^[	ESC	\e	溢出	43	+	59	;	75	K	91	[	107	k	123	{	
1100	C	12	♀	^L	FF	\f	换页	28	∟	^\	FS		文件分隔符	44	,	60	<	76	L	92	\	108	l	124	|	
1101	D	13	♪	^M	CR	\r	回车	29	↔	^]	GS		组分隔符	45	-	61	=	77	M	93	]	109	m	125	}	
1110	E	14	♫	^N	SO		移出	30	▲	^^	RS		记录分隔符	46	.	62	>	78	N	94	^	110	n	126	~	
1111	F	15	☼	^O	SI		移入	31	▼	^-	US		单元分隔符	47	/	63	?	79	O	95	_	111	o	127	⌂	^Backspace 代码：DEL

注：表中的ASCII字符可以用“Alt + 小键盘上的数字键”方法输入。 2013/08/08

2、重要字符的ASCII码值

```
'\0' ASCII值是 0
'0'  ASCII值是 48
'A'  ASCII值是 65
'a'  ASCII值是 97
```

3、字符的输出

```
printf("%c",ASCII值);
putchar(ASCII值);
```

3、字符的输入

```
char ch;
scanf("%c",&ch);
ch = getchar();
```

4、输入字符时要注意的问题

当先输入数值型数据(整数形、浮点型)，再输入字符型数据时，上一次的输入会残留一个'\n'或空格，影响字符型数据的输入，有以下三种解决方法。

```
方法1：增加一个空白的字符输入语句。
scanf("%*c");
getchar();

方法2：在%c的前面增加一个空格
scanf(" %c");

方法3：修改输入缓冲区
stdin->_IO_read_ptr = stdin->_IO_read_end;
```

练习：输入n个字符，统计每个字符出现的次数。

```
#include <stdio.h>

int main(int argc,const char* argv[])
{
    int n , arr[128] = {};
    scanf("%d",&n);

    //getchar();
    //scanf("%*c");
    stdin->_IO_read_ptr = stdin->_IO_read_end;

    char ch;
    for(int i=0; i<n; i++)
    {
        //scanf(" %c",&ch);
        ch = getchar();
        arr[ch]++;
    }

```

```
    for(int i=0; i<128; i++)
    {
        if(arr[i])
            printf("%c %d\n",i,arr[i]);
    }
}
```

5、判断字符类型的函数：

函数名	函数功能
isalnum()	当字母或数字字符时, 返回真值
isalpha()	当字母字符时, 返回真值
iscntrl()	当控制字符时, 返回真值
isdigit()	当数字字符时, 返回真值
isgraph()	当非空格可打印字符时, 返回真值
islower()	当小写字母字符时, 返回真值
isprint()	当可打印字符时, 返回真值
ispunct()	当标点字符时, 返回真值
isspace()	当空格字符时, 返回真值
isupper()	当大写字母字符时, 返回真值
isxdigit()	当十六进制字符时, 返回真值

十七、输入、输出缓冲区

1、输出缓冲区

当我们使用标准库的输出系列函数打印数据到屏幕，数据并不会立即显示到屏幕上，而先存储到一块内存中，我们把这块内存称为输出缓冲区，等满足相关条件后，再从缓冲区中显示到屏幕。

2、数据从输出缓冲区到达文件的条件有

1、从输出状态切换到输入状态。

2、缓冲区满了，1k=1024个字节，系统会把缓冲区中所有数据一起显示到屏幕了。

3、程序结束时，系统会把缓冲区中所有数据一起显示到屏幕了。

4、遇到'\n'时，'\n'前面的数据会立即显示到屏幕上。

5、调用fflush(stdout) 强制刷新，会把立即缓冲区中所有数据一起显示到屏幕了。

3、输入缓冲区

当们们从终端输入数据给程序时，系统并没有立即把数据交给程序读取，而先存储到了一块内存中，我们这块内存称为输入缓冲区，直到我们按下Enter键时，系统才会把缓冲区中的数据给程序读取。

当我们输入的数据过多，或者类型不匹配，标准的输入系列函数就会读取失败，或只读取一部分，剩余的数据就会残留缓冲区中，影响后续数据的输入，当我们发现这情况情况后，应先清理输入缓冲区，后续的数据才能正常输入。

4、清理输入缓冲区的方法

```
方法1:
while('\n' != getch());

方法2:
scanf("%*[^\n]");
scanf("%*c");

方法3:
stdin->_IO_read_ptr = stdin->_IO_read_end;

注意:
    1、如果输入缓冲区中没有垃圾数据，使用方法1与方法2需要你手动输入一些垃圾数据，程序才能继续执行。
    2、方法3只能在Linux系统下使用。
```

十八、字符串

1、什么是串型结构

由若干个相同类型的数据组成顺序表（数组），在数据的末尾有一个结束标志，在使用这种数组时，可以不关心数组的长度。

2、什么是字符串

由字符类型组成的串型结构，它的结束标志是'\0'，使用它可以存储单词、句子、文章、汉字等更丰富的信息，一般使用char类型的数组存储。

```
// 定义字符串时，要为'\0'预留位置
char arr1[] = {'H','e','l','l','o','\0'};
char arr2[10] = {'H','e','l','l','o'};
```

3、字符串字面值

1、"由双引号包括着的若干个字符"

2、它是以常量字符数组的形式存在，末尾隐藏着一个'\0'。

3、它们会被存储在text内存段，一旦强行修改就会出现段错误。

4、使用指针指向字符串字面值时，一定要用const加以保护，防止出现段错误，宁可出现编译时的错误，也不要出现运行时的错误。

5、编译器会优化它的存储，相同的字符串字面值，只会存储一份在text内存段中。

6、最常用的是用它给字符数组初始化，char arr[] = "hello" 编译器会自动拷贝字符串到数组的内存中（包括'\0'），完成初始化就有了两份字符串存储在内存中，一份存储在stack、data，另一份还存储在text。

注意：使用字符串字面值给字符数组赋值，只能在定义字符数组时使用，这是编译器帮忙完成拷贝的，在完成字符数组的定义后，只能使用strcpy函数对字符串进行赋值。

```
#include <stdio.h>

int main(int argc,const char* argv[])
{
```

```
    char* p1 = "world";
    char* p2 = "world";
    printf("%p %p\n",p1,p2); // 相同的字符串字面值，只会存储一份在内存中

    int len = sizeof("hello");
    printf("%d\n",len); // 结果是6,所以字符串字面值是数组

    char* str = "hello";
    printf("%p\n",str); // 该地址是text内存段范围
    str[0] = 'H';   // 会出现段错误,使用指针变量指向字符串字面值时应该用const保护一下

    printf("%s\n",str);

    char arr[] = "hello";
    printf("%s\n",arr);
}
```

4、字符串的输出

```
printf("%s",字符串的首地址);
puts(字符串的首地址); // 输出完字符串后会再输出一个\n
```

5、字符串的输入

```
scanf("%s",存储字符串的首地址);
缺点：不能输入带空格的字符串

char *gets(char *s);
缺点：从终端读取数据，直到遇到\n，它不关心存储字符串的空间有多大，所以可能产生段错误、脏数据，官方不建议使用该函数。

char *fgets(char *s, int size, FILE *stream);
功能：从指定的文件中读取不超过size-1个字符(它会为'\0'预留位置)。
stream：数据的来源，写stdin即可
缺点1：如果输入的字符不足size-1个，它会连'\n'一起读入。
缺点2：如果输入的字符超过size-1个，fgets只会从输入缓冲区中读取size个字符，剩余的数据会残留输入缓冲区中影响后续数据的输入。

解决方法：
int main(int argc,const char* argv[])
{
    char str[20];
    fgets(str,20,stdin);

    int len = -1;
    while(str[++len]);

    if('\n' != str[len-1])
        while('\n' != getch());
    else
        str[len-1] = '\0';

}

char* get_str(char* str,size_t size)
{
```

```
    assert(NULL != str);
    size_t len = strlen(fgets(str,size,stdin));

    if('\n' == str[len-1])
        str[len-1] = '\0';
    else
        while('\n' != getchar());
    return str;
}
```

6、操作字符串的常用函数

```
size_t strlen(const char *s);
功能：计算字符串的长度，不包括'\0'

char *strcpy(char *dest, const char *src);
功能：把字符串src拷贝到dest处

int strcmp(const char *s1, const char *s2);
功能：按字典序比较两个字符串
    s1 > s2 返回正数
    s1 < s2 返回负数
    s1 == s2 返回0

char *strcat(char *dest, const char *src);
功能：把src字符串追加到dest的末尾
```

十九、电子通讯录作业

1、联系人信息：

姓名、性别、联系电话

2、功能

1、添加联系人

2、删除联系人

3、修改联系人信息

4、查询联系人

5、显示所有联系人

6、该通讯录最多只存储100个联系人

3、参考代码

```
#include <stdio.h>

char name[100][20];
char sex[100];
char phone[100][12];

int main()
{
    for(;;)
```

```
    {
        // 功能提示
        puts("*****欢迎指针xxx通讯录*****");
        puts("1、添加联系人");
        puts("2、删除联系人");
        puts("3、修改联系人信息");
        puts("4、查询联系人");
        puts("5、显示所有联系人");
        puts("6、退出");
        puts("--------------");
        printf("请输入指令:")

        switch(getchar())
        {
            case '1': // 调用添加联系人功能函数 break;
            case '2': // 调用删除联系人功能函数 break;
            case '3': // 调用修改联系人信息功能函数 break;
            case '4': // 调用查询联系人功能函数 break;
            case '5': // 调用显示所有联系人功能函数 break;
            case '6': return 0;
        }
    }
}
```

二十、C语言的堆内存管理

C语言中没有管理堆内存的语句，C标准库中提供一套管理堆内存的函数，这些函数底层封装了各操作系统的堆内存管理接口，所以可以跨平台使用，这些函数声明在 **stdlib.h** 头文件中。

1、malloc函数

```
void *malloc(size_t size);
功能：向malloc申请位size字节的堆内存块
size：
    要申请的内存块字节数
    如果申请数组形式的内存块，size=sizeof(数组元素类型)*数组长度
返回值：
    如果申请成功，则返回内存块首地址
    失败申请失败，则返回NULL，现在有堆内存无法满足size个字节的需求

// 如果size的值过大，就会申请失败
int main(int argc,const char* argv[])
{
    int* p = malloc(0xffffffff);
    printf("%p\n",p); // (nil)
}


// 向malloc申请n字节内存
int main(int argc,const char* argv[])
{
    int* p = malloc(4);
    printf("请输入一个整数：");
    scanf("%d",p);
    printf("%d\n",*p);
}

```

```c
// 向malloc申请一块 一维数组内存
int main(int argc,const char* argv[])
{
    int len;
    printf("请输入数组的长度：");
    scanf("%d",&len);

    int* arr = malloc(sizeof(int)*len);
    for(int i=0; i<len; i++)
    {
        printf("%d ",arr[i]);
    }
}

// 向malloc申请一块 二维数组内存
int main(int argc,const char* argv[])
{
    int row,col;
    printf("请输入二维数组的行数和列数：");
    scanf("%d%d",&row,&col);

    int (*arr)[col] = malloc(sizeof(int)*row*col);
    for(int r=0; r<row; r++)
    {
        for(int c=0; c<col; c++)
        {
            printf("%d ",arr[r][c]);
        }
        printf("\n");
    }
}
```

2、使用malloc函数要注意的问题

1、使用malloc申请到的内存块，里面的内容是不确定的，malloc不会帮我们初始化，可以使用bzero,memset函数进行初始化。

2、如果size等于0，返回NULL或唯一个的地址，并且该地址可以通过free释放而不出错，但不能使用它指向的内存。

3、calloc函数

```c
void *calloc(size_t nmemb, size_t size);
功能：申请nmemb个size个字节的内存块，专门用于申请数组型的内存块。
nmemb：数组的长度
size：数组元素的字节数
返回值：与malloc相同
注意：使用calloc申请的内存块，所有字节会被初始化0。

int main(int argc,const char* argv[])
{
    int len;
    printf("请输入数组的长度：");
    scanf("%d",&len);


    int* arr = calloc(len,sizeof(int));
```

```
    for(int i=0; i<len; i++)
    {
        // 输出结果肯定是0
        printf("%d ",arr[i]);
    }
}
```

4、使用calloc函数要注意的问题

1、calloc所申请也是一块连续的内存块，所以nmemb和size的参数位置可以调换，就相当于calloc内部调用了malloc函数，只是比malloc多了初始化步骤，而且比malloc的可读性更高。

2、malloc比calloc申请内存的速度快，或者使用malloc+bzero配合。

练习：实现自定义的calloc函数，底层调用malloc函数实现。

```
void* my_calloc(size_t nmemb,size_t size)
{
    char* ptr = malloc(nmemb*size);
    for(int i=0; i<nmemb*size; i++)
    {
        ptr[i] = 0;
    }
    return ptr;
}

int main(int argc,const char* argv[])
{
    int len;
    printf("请输入数组的长度：");
    scanf("%d",&len);

    int* arr = my_calloc(len,sizeof(int));
    for(int i=0; i<len; i++)
    {
        // 输出结果肯定是0
        printf("%d ",arr[i]);
    }
}
```

5、free函数

```
void free(void *ptr);
功能：释放堆内存
ptr：要释放的内存块的首地址，它必须是malloc、calloc函数的返回值

int main(int argc,const char* argv[])
{
    int len = 20;
    int* arr = malloc(sizeof(int)*len);
    for(int i=0; i<len; i++)
    {
        arr[i] = 0x01020304;
        printf("%d ",arr[i]);
    }
    printf("\n");

```

```
    // 释放掉使用权，并破坏一部分内容
    free(arr);

    // 非法使用
    for(int i=0; i<len; i++)
    {
        printf("%d ",arr[i]);
    }
    printf("\n");
}
```

6、使用free函数要注意的问题

1、free释放的是使用权，只破坏内存块的一部分内存，大部分数据还在，这样设计的原因是释放速度比较快，就像在硬盘上删除文件一样，只是把存储文件那片区域的使用释放旧，数据还存储在磁盘上，这也是我们能进行数据恢复的原因。

2、free的参数可以是空指针，不会出现错误，也不会执行任何操作，这也是空指针比野指针安全的原因。

3、如果内存被重复释放则会出现"double free or corruption (fasttop)"，程序会异常停止，所以在第一次释放内存后，要把与堆内存配合的指针及时的赋值为空，防止重复释放产生的错误。

7、realloc函数

```
void *realloc(void *ptr, size_t size);
功能1：把已有的堆内存块调小
    ptr是malloc、calloc、realloc的返回值，也就已有堆内存块首地址
    size < oldsize
    此时不需要关心realloc的返回值
int main(int argc,const char* argv[])
{
    int* p1 = malloc(1024);
    p1 = realloc(p1,10);
    int* p2 = malloc(1024);
    printf("%p %p\n",p1,p2);
}

功能2：把已有的堆内存块调大
    ptr是malloc、calloc、realloc的返回值，也就已有堆内存块首地址
    情况1：如果ptr后续的内存没有被占用，realloc会在ptr的基础上进行扩大
    情况2：如果ptr后续的内存已经被占用，realloc会重新分配一块符合要求的内存块，并把ptr上的内容拷贝到新的内存块，然后释放ptr，再返回新内存块的首地址
    使用此功能时，我们必须重新接收realloc函数的返回值，我们无法预料realloc执行的是情况1还是情况2。
// 情况1
int main(int argc,const char* argv[])
{
    int* p1 = malloc(40);
    int* p2 = realloc(p1,80);
    // 此时p1 == p2,是在p1的基础上直接扩大的
    printf("%p %p\n",p1,p2);
}
// 情况2
int main(int argc,const char* argv[])
{
    int* p1 = malloc(40);
```

```
    for(int i=0; i<10; i++)
        p1[i] = i+1;

    // 占了p1r的后续位置，realloc就不能在p1的基础上直接扩展了
    int* tmp = malloc(4);

    int* p2 = realloc(p1,80);
    // 此时p1 != p2,重新分配了一块内存,先把p1的内容拷贝新内存块，并把p1释放掉了,然后返回新内存块的地址
    printf("%p %p\n",p1,p2);
    for(int i=0; i<10; i++)
    {
        printf("%d %d\n",p1[i],p2[i]);
    }
}

功能3：释放内存
    ptr是malloc、calloc、realloc的返回值
    0==size，此时realloc的功能就相当于free

功能4：申请内存
    NULL==ptr，0<size
    此时的功能就相当于malloc
```

8、使用realloc函数要注意的问题

虽然realloc具有释放和申请堆内存的功能，但我们一般不使用，而是直接使用malloc和free，主要使用的是realloc的调整内存块大小的功能。

9、总结

使用堆内存只需要掌握malloc和free函数即可，对于calloc和realloc函数了解即可。

二一、堆内存越界

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc,const char* argv[])
{
    char* ptr = malloc(1);
    // 只要越界的不超过135160,就不会出现段错误
    printf("%c\n",ptr[135160]);
}

```

1、malloc管理堆内存的机制

1、当程序首次向malloc申请内存时，此时malloc手里没有堆内存可分配，malloc会向操作系统申请堆内存，操作系统会一次性分配33页内存交给malloc管理(一页内存=4096个字节)，之后再向malloc申请内存时，malloc会从这33页内存中分配给调用者。

2、使用malloc申请的每个内存块前面会有4~12个字节的空隙，malloc会根据所申请的内存块的大小自动调整空隙的大小。

3、内存块前面的空隙有两部分：

空隙的前0~8字节：用于内存对齐

空隙的末尾4字节：也就是内存块前面的4字节，存储首malloc的管理信息，这块信息被破坏会影响后续malloc、free、printf、scanf函数的使用。

```
int main(int argc,const char* argv[])
{
    int* p1 = malloc(8);
    p1[-1] = 0; // 管理信息会影响p1的释放
    p1[0] = 1234; // 申请的4字节内存块
    p1[1] = 5678; // 空闲字节
    p1[2] = 6666; // 空闲字节
    p1[3] = 0;    // 管理信息,一旦被破坏会影响后续malloc函数的使用
    int* p2 = malloc(8);
    int* p3 = malloc(12);
    int* p4 = malloc(4);
    printf("%p \n",p1);
    printf("%p \n",p2);
    printf("%p \n",p3);
    printf("%p \n",p4);
}
```

2、为什么是135160会出现段错误

操作系统交给malloc33页内存(135168个字节)，可访问的范围是0~135167，malloc会预留8个字节的空隙，返回给程序的是33页内存的第9个字节的地址（33页内存还剩135160个字节），所以可访问的范围是0~135159，只要在这个范围就不会出现段错误。

3、这种分配机制的优点

1、避免了频繁打扰操作系统，而影响操作系统的速度。

2、段错误产生的原因是被操作系统发现非法使用内存，所以我们使用malloc分配的内存越界时，只要不超过33页范围就不会产生段错误。

4、使用堆内存越界的后果：

1、越界使用的是空隙的空闲字节，一切正常，可以安全访问。

```
int main()
{
    int* p = malloc(4);
    p[0] = 123; // 申请到的内存块
    p[1] = 456; // 空闲
    p[2] = 789; // 空闲
}
```

2、越界破坏了malloc的管理信息，会影响后续malloc、free、scanf、printf函数的使用。

```
int main()
{
    int* p = malloc(4);
    p[3] = 0;   // 存储着malloc的管理信息，后续无法继续申请堆内存
    p[-1] = 0;  // 存储着malloc的管理信息，p内存块无法释放
}
```

3、越界使用malloc还未分配出去的内存，虽然不会产生段错误，但后续malloc把它分配出去后，可能会产生脏数据。

```
int main(int argc,const char* argv[])
{
    int* p1 = malloc(4);
    p1[4] = 6666;
    printf("%d\n",p1[4]);   // 6666
    printf("%d\n",p1[4]);   // 随机值
}

int main(int argc,const char* argv[])
{
    int* p1 = malloc(4);
    p1[4] = 6666;

    int* p2 = malloc(4); // malloc把p1[4]位置内存分配给了p2
    *p2 = 123456789;
    printf("%d\n",p1[4]);
    printf("%d\n",*p2);
}
```

4、超出33页范围，就产生段错误。

```
int main(int argc,const char* argv[])
{
    char* ptr = malloc(1);
    // 只要越界的不超过135160,就不会出现段错误
    printf("%c\n",ptr[135160]);
}

int main(int argc,const char* argv[])
{
    int* p1 = malloc(4);
    printf("%d\n",p1[33790]);

}
```

二二、内存碎片

1、什么是内存碎片

已经释放了使用的权的内存，但无法被malloc再次分配出去，这种内存叫内存碎片。

```
int* p1 = malloc(4);
int* p2 = malloc(4);
int* p3 = malloc(4);

free(p2);

// 此时p2已经被释放，但无法再次分配给p4，p2就是内存碎片，如果后续不再分配较小的内存块，p2可能一直接内存碎片
int* p4 = malloc(16);

// 此时p2就有可能被再次分配出来，它就不是内存碎片，
int* p5 = malloc(4);
```

```

// 如果p1或p3被释放，p2就不是内存碎片了
free(p1);
free(p3);
```

2、内存碎片产生的原因

内存和释放、分配时间、大小不协调导致的。

一块内存碎片，只是短时间内是碎片，过一段时间它可能就不是碎片了，所以内存碎片不是绝对的。

3、如何减少内存碎片

前提：内存碎片只能尽量减少，无法杜绝。

1、尽量使用栈内存（要了解栈内存的特性，要知道栈内存的使用上限，ulimit -s 、ulimit -s ）。

2、尽量分配大块内存自己管理。

```
int* p1 = malloc(16);
int* p2 = p1+1;
int* p3 = p1+2;
int* p4 = p1+3;
```

3、按照分配的顺序，逆序释放，把堆内存当栈进行管理。

```
int* p1 = malloc(4);
int* p2 = malloc(4);
int* p3 = malloc(4);
int* p4 = malloc(4);

free(p4);
free(p3);
free(p2);
free(p1);
```

4、内存碎片整理

```
int* p1 = malloc(4);
int* p2 = malloc(4);
int* p3 = malloc(4);

memcpy(p2,p3,4);
swap(p2,p3);
free(p2);
```

二三、内存泄漏

1、什么是内存泄漏

内存已经不再使用，但无法被释放的内存叫内存泄漏。

但这不是最严重的，严重的是反复的内存泄漏，例如：一个函数执行需要分配一块堆内存，等它执行完毕后堆内存没有被释放，等再次调用这个函数时，它又重新分配堆内存，又没有释放，反复这样会导致可用的内存越来越少，系统、程序会变得越来越慢、卡、死机。

```c
void func(void)
{
    int* p = malloc(40);
    for(int i=0; i<10; i++)
    {
        p[i] = rand() % 100;
        printf("%d ",p[i]);
        if(条件)
            return;
    }
    printf("\n");
    free(p);
}

int main()
{
    for(;;)
    {
        ...
        func();
        ...
    }
}
```

2、产生内存泄漏的原因

1、只写的内存分配语句，而忘记写内存释放语句，可能是粗心大意，也可能是以为别人会释放。

2、写了内存释放语句，但由于执行流程、执行条件设计有问题，导致释放语句没有执行。

3、与堆内存配合的指针被破坏，改变了指向，导致free语句执行无效。

3、如何减少内存泄漏

1、按规则分配、释放内存：

自用：谁申请谁释放，分配语句和释放语句成对出现。

共用：谁知道该释放谁释放，项目组中负责分配和负责释放的人要进行对接。

2、封装malloc和free函数，记录每一次的分配和释放的内存块地址，通过对比记录，就可以发现是否有内存泄漏。

```c
void my_free(void* ptr)
{
    printf("free %p\n",ptr);
    free(ptr);
}

void* my_malloc(size_t size)
{
    void* ptr = malloc(size);
    printf("malloc %p\n",ptr);
    return ptr;
}
```

3、使用const保护与堆内存配合的指针变量，防止指针被破坏。

```c
int* const p = malloc(4);

p = &num; // 编译会出错

free(p); // 此时释放的肯定是堆内存
```

4、检查内存泄漏的工具

```bash
# 安装该工具的命令
sudo apt install valgrind

#使用该工具检测程序：
valgrind --tool=memcheck --leak-check=yes ./a.out
```

5、内存泄漏和内存碎片的危害

前提：当程序结束时，操作系统会把分配它的所有资源全部回收(包括系统分配给程序的堆内存)，所以当程序结束时，内存碎片和内存泄漏就会消失，这也是为什么软件、系统重启能解决很多问题。

注意：服务器端的程序一般需要7*24小时运行，不能随意结束。

客户端的程序可以随意的关闭、重启系统和软件，所以即使发生内存碎片和内存泄漏，也问题不大，但如果是服务端的程序，即使只有少量的内存泄漏和内存碎片，长年累月下来也会导致系统可用的内存越来越少，系统、程序会变得越来越慢、卡、死机。

二四、结构

1、什么是结构

是一种由程序员设计的复合数据类型，它由若干个其它类型的成员组成，用于统一描述事物的各项属性。

使用各类型的变量也可以描述事物的各项属性（如：通讯录项目），但使用麻烦且容易出错，没有使用结构方便，安全性高、统一性高，同时结构也是面向对象编程的基础。

2、如何设计结构

```c
struct 结构名 // 结构名一般首字母大写
{
    成员类型 成员名;
    ...
};

// 为联系人设计的结构
struct Contacts
{
    char name[20];
    char sex;
    char phone[12];
};

注意：结构设计完成后，仅仅是一种数据类型，必须定义结构变量、结构数组、分配堆内存才能使用。
```

3、如何使用结构

```
// 在C语言中，使用结构类型时 struct 关键字不能省略
struct 结构名 结构变量名;

struct 结构名 结构数组名[n];

struct 结构名* 结构指针变量名 = malloc(sizeof(struct 结构名)*数量);

void func(struct 结构名* 指针)
{// 一般结构变量的占用字节较多，所以使用指针进行传参提高传参效率，也可以使用const保护结构变量

}
```

4、初始化结构变量

```
// 顺序初始化,数据要与成员的顺序一一对应。
struct 结构名 结构变量名 = {v1,v2,v3,...};
struct 结构名 结构数组[n] = {
    {v1,v2,v3,...},
    {v1,v2,v3,...},
    ...
};

// 指定成员初始化
struct 结构名 结构变量名 = {
    .成员名1 = 初始化数据1,
    .成员名2 = 初始化数据2,
    ...
};

struct 结构名 结构数组[n] = {
    {.成员名1 = 初始化数据1,.成员名2 = 初始化数据2,...},
    {.成员名1 = 初始化数据1,.成员名2 = 初始化数据2,...},
    {.成员名1 = 初始化数据1,.成员名2 = 初始化数据2,...},
    ...
};


```

5、访问结构成员

```
// 结构变量.成员名;
// 结构指针->成员名;

#include <stdio.h>

// 设计联系人结构类型
struct Contacts
{
    char name[20];
    char sex;
    char phone[12];
};

```

```c
void show_contacts(const struct Contacts* cts)
{
    printf("%s %c %s\n",cts->name,cts->sex,cts->phone);
}

int main(int argc,const char* argv[])
{
    // 定义结构变量并顺序初始化
    struct Contacts cts = {"李经理",'w',"13949953245"};

    // 访问成员
    //printf("%s %c %s\n",cts.name,cts.sex,cts.phone);
    show_contacts(&cts);



    // 定义结构变量并指定初始化成员
    struct Contacts cts1 = {
        .name = "孙主管",
        .phone = "15979631133"
    };

    // 访问成员
    //printf("%s %c %s\n",cts1.name,cts1.sex,cts1.phone);
    show_contacts(&cts1);
}

```

6、使用typedef重定义简短的类型名

在C语言中，struct 结构名 才是完整的数据类型，但使用时比较麻烦，可以使用typedef给结构重定义简短的类型名。

```c
// 结构设计完成后重定义
typedef struct 结构名 结构类型名;

// 设计结构时重定义
typedef struct 结构名
{
    成员类型 成员名;
    ...
}结构名;
```

```c
#include <stdio.h>

// 设计联系人结构类型，设计结构时重定义类型名
typedef struct Contacts
{
    char name[20];
    char sex;
    char phone[12];
}Contacts;

// 设计结构完成后，重定义简短的类型名，一般使用结构名作为类型名
// typedef struct Contacts Contacts;

```

```c
void show_contacts(const Contacts* cts)
{
    printf("%s %c %s\n",cts->name,cts->sex,cts->phone);
}

int main(int argc,const char* argv[])
{
    // 定义结构变量并顺序初始化，tyepdef重定义后可以省略struct关键字
    Contacts cts = {"李经理",'w',"13949953245"};

    // 访问成员
    //printf("%s %c %s\n",cts.name,cts.sex,cts.phone);
    show_contacts(&cts);
}
```

练习：设计一个教师结构体(姓名，工龄，工号，科目，...),定义教师结构变量，使用scanf从终端输入各成员的值，然后使用printf显示结构变量。

```c
#include <stdio.h>

typedef struct Teacher
{
    char name[20];
    char work_age;
    int work_id;
    char subject[50];
}Teacher;

int main(int argc,const char* argv[])
{
    Teacher tch;
    printf("请输入教师的姓名、工龄、工号、科目");
    scanf("%s %hhd %d %s",tch.name,&tch.work_age,&tch.work_id,tch.subject);

    printf("%s %hhd %d %s\n",tch.name,tch.work_age,tch.work_id,tch.subject);
}
```

二五、结构的总字节数

1、为什么要学会计算结构的总字节数

1、结构变量的总字节数 >= 所有成员的字节数总和

2、结构成员的顺序会影响结构的总字节数

3、了解结构总字节数计算规则，可以通过合理安排成员的顺序达到节约内存的目的

4、计算机为了提高结构成员的访问速度，会在结构变量的内部填充一些空闲的内存，这叫做内存对齐和补齐

5、在笔记题中考的很多，希望大家重视。

2、内存对齐

假定从0字节排列结构的第一个成员，之后所有成员的起始字节数，必须是成员本身字节数的整数倍，如果不是则填充一些空闲字节。

```
#include <stdio.h>

typedef struct Data
{
    char ch;     // 0
                 // 1 2 3 空闲了3个字节用于内存对齐
    int num;     // 4 5 6 7

}Data;

int main(int argc,const char* argv[])
{
    Data d;
    printf("%p %p\n",&d.ch,&d.num); // 这个地址之间距离是4字节，ch占用了1字节，所以空了3个字节
    printf("%d\n",sizeof(d));
}
```

3、内存补齐

结构的总字节数必须是它最大成员字节数的整数倍，如果不是则在结构的末尾填充一些空闲字节。

```
typedef struct Data
{
    char ch;  // 0
              // 1 2 3
    int num;  // 4 5 6 7
    short sh; // 8 9 目前为止使结构体总字节数是10字节，最大成员是4字节，10不是4的整数倍，所以在末尾填充2字节
              // 10 11
}Data

int main(int argc,const char* argv[])
{
    Data d;
    printf("%p %p\n",&d,&d.sh);

    printf("%d\n",sizeof(d));
}
```

注意：在32位系统下，内存对齐、内存补齐字节数是有上限的，超过上限按4字节计算。

```
// 在Linux32位系统下，超过4字节按4字节计算。
typedef struct Data
{
    char ch;     // 0
                 // 1 2 3
    double num; // 4 ~ 11
    short sh;    // 12 13
                 // 14 15
```

```
}Data;

int main(int argc,const char* argv[])
{
    printf("%d\n",sizeof(Data)); // 结果是16字节
}

```

```
#include <stdio.h>
// 在Windows32位系统下，超过8字节按4字节计算。
typedef struct Data
{
    char ch;            // 0
                        // 1 2 3
    long double num;    // 4 ~ 15
    short sh;           // 16 17
                        // 18 19

}Data;

int main(int argc,const char* argv[])
{
    printf("%d\n",sizeof(Data));    // 结果是20字节
}
```

```
#include <stdio.h>

// 在Windows32位系统下,未超过8字节，按8字节对齐补齐
typedef struct Data
{
    char ch;    // 0
                // 1 2 3 4 5 6 7
    double num; // 8 ~ 15
    short sh;   // 16 17
                // 18 19 20 21 22 23

}Data;

int main(int argc,const char* argv[])
{
    printf("%d\n",sizeof(Data)); // 结果是24字节
}
```

```
#include <stdio.h>
// Windows64位系统和Linux64位系统，都按成员的字节数计算内存对齐、内存补齐
typedef struct Data
{
    char ch;    // 0
                // 1 2 3 4 5 6 7 8 9 11 12 13 14 15
    long double num;    // 16 ~ 31
    short sh;   // 32 33
                // 34 35 36 37 38 39 40 41 42 42 44 45 46 47

}Data;

```

```c
int main(int argc,const char* argv[])
{
    printf("%d\n",sizeof(Data));
}
```

4、long类型的字节数

Linux32位系统 4字节

Linux64位系统 8字节

Windows32位系统 4字节

Windows64位系统 4字节

二六、联合

1、联合与结构的区别

也是一种由程序员设计的复合数据类型，使用语法与结构一模一样，与结构不同的是联合中的所有成员共用一块内存，所以只要有一个成员的值发生变化，其它成员的也会跟着一起变化。

```c
union 联合名
{
    成员类型 成员名;
    ...
};

#include <stdio.h>
typedef union Data
{
    char ch;
    int num;
    long ld;
}Data;

int main(int argc,const char* argv[])
{
    printf("%d\n",sizeof(Data));
    Data d;
    printf("%p %p %p\n",&d.ch,&d.num,&d.ld);
    d.ch = 123;
    printf("%hhd %d %ld\n",d.ch,d.num,d.ld);
    d.num = 6666;
    printf("%hhd %d %ld\n",d.ch,d.num,d.ld);
    d.ld = 3456;
    printf("%hhd %d %ld\n",d.ch,d.num,d.ld);

}
```

2、联合的总字节数

由于联合的所有成员共用一块内存，所有成员是天然对齐的，不需要考虑内存对齐，但要考虑内存补齐。

情况1：所有联合的成员都是基本类型，则联合的总字节数就是最大成员的字节数。

情况2：如果联合的成员有数组类型，则联合的总字节数应该是最大成员的整数倍。

```c
#include <stdio.h>

typedef union Data1
{
    char ch;
    int num;
    long ld;
}Data1;

typedef union Data2
{
    long num;
    char arr[5];
}Data2;

int main(int argc,const char* argv[])
{
    printf("%d\n",sizeof(Data1)); // 结果是4
    printf("%d\n",sizeof(Data2)); // 结果是8
}
```

3、使用联合的意义

1、使用少量的内存对应若干个标识符，只要不同时使用联合的成员，就不会冲突，能大大节约内存，在早期计算机内存比较小时，该技术使用较多，现在随着计算机内存越来越大已经基本不再使用。

2、联合可以对一块内存进行不同格式的解释，所以在网络通信时还存在着少量的使用。

二七、大端系统和小端系统

假定有一个int类型变量，它的4字节的内存地址分别是：

```c
int num;

0xbf9f3828  // 低位地址
0xbf9f3829
0xbf9f382a
0xbf9f382b  // 高位地址
```

假定有一个十六进制的整数：0xa1b2c3d4

```c
0xa1    //高位数据
0xb2
0xc3
0xd4    //低位数据
```

1、大端系统

低位数据存储高位地址，或者说是高位数据存储在低位地址，一般大型的服务器、网络设备采用的是大端系统，所以大端格式也叫网络字节序。

```
int num = 0xa1b2c3d4;

0xbf9f3828  存储的是0xa1
0xbf9f3829  存储的是0xb2
0xbf9f382a  存储的是0xc3
0xbf9f382b  存储的是0xd4
```

2、小端系统

低位数据存储在低位地址，或者说高位数据存储在高位地址，一般的个人计算机采用的是小端系统。

```
int num = 0xa1b2c3d4;

0xbf9f3828  存储的是0xd4
0xbf9f3829  存储的是0xc3
0xbf9f382a  存储的是0xb2
0xbf9f382b  存储的是0xa1
```

注意：数据存储的是大端格式还是小端格式是由计算机的CPU决定的。

练习：使用联合判断当前系统是大端还是小端。

```c
#include <stdio.h>

typedef union Data
{
    unsigned char ch;
    int num;
}Data;

int main(int argc,const char* argv[])
{
    Data d;
    d.num = 0xa1b2c3d4;

    if(d.ch == 0xd4)
        puts("小端系统");
    else
        puts("大端系统");
}

int main(int argc,const char* argv[])
{
    int num = 0xa1b2c3d4;
    if(*(unsigned char*)&num == 0xd4)
        puts("小端系统\n");
    else
        puts("大端系统\n");
}
```

二八、枚举

是一种值受限的整数类型，由程序员设置它的值的范围。

1、设计枚举

```
enum 枚举名
{
    枚举值1,
    枚举值2,
    枚举值3,
    ...
};

注意：使用标识符作为枚举值

// enum与struct、union一样，使用typedef重定义省略enum关键字
typedef enum 枚举名
{
    枚举值1,
    枚举值2,
    枚举值3,
    ...
}枚举名;
```

2、定义枚举变量

enum 枚举名 枚举变量;

1、理论上枚举变量只能使用枚举值赋值，这样可以提高代码的可读性和安全性。

2、C语言编译器为了提高编译速度，不会检查枚举变量的赋值，全靠程序员的自觉（枚举变量就是int类型变量）。

3、C++编译器类型检查比较严格，所以使用C++编译器编译C代码时，枚举变量只能由枚举值赋值、比较。

```
#include <stdio.h>

enum DirectionKey
{
    Up,
    Down,
    Right,
    Left
};

int main(int argc,const char* argv[])
{
    enum DirectionKey key;
    key = 123;

    printf("%d\n",key);
}
```

3、枚举值

1、第一个枚举值的默认值是0，之后的枚举值逐渐递增+1。

```c
#include <stdio.h>

enum DirectionKey
{
    Up,
    Down,
    Right,
    Left
};

int main(int argc,const char* argv[])
{
    enum DirectionKey key;
    key = Up;
    printf("%d\n",key); // 0
    key = Down;
    printf("%d\n",key); // 1
    key = Right;
    printf("%d\n",key); // 2
    key = Left;
    printf("%d\n",key); // 3
}
```

2、可以使用=设置枚举值，没有进行设置的枚举值是上一个枚举值递增+1。

```c
#include <stdio.h>

enum DirectionKey
{
    Up = 123,
    Down,
    Right = 456,

    Left
};

int main(int argc,const char* argv[])
{
    enum DirectionKey key;
    key = Up;
    printf("%d\n",key); // 123
    key = Down;
    printf("%d\n",key); // 124
    key = Right;
    printf("%d\n",key); // 456
    key = Left;
    printf("%d\n",key); // 457
}
```

3、枚举值可以单独使用，这种用法可以给没有意义的字面值数据取一个有意义的名字，这样可以提高代码的可读取性，也可以定义匿名的枚举，只使用枚举值。

```c
#include <stdio.h>

enum
{
    Up = 123,
    Down,
    Right = 456,
    Left
};

int main(int argc,const char* argv[])
{
    printf("%d %d %d %d\n",Up,Down,Right,Left);
}
```

4、枚举值是常量，所以可以与switch语句配合使用，枚举值可以写在case的后面。

```c
#include <stdio.h>
#include <getch.h>

enum
{
    Up = 183,
    Down,
    Right,
    Left
};

int main(int argc,const char* argv[])
{
    for(;;)
    {
        switch(getch())
        {
            case Up: puts("上"); break;
            case Down: puts("下"); break;
            case Right: puts("右"); break;
            case Left: puts("左"); break;
        }
    }
}
```

5、枚举值的作用域是全局的，所以它不能与全局变量、函数、结构、联合重名。

```c
#include <stdio.h>
#include <getch.h>

int num;
enum
{
    Up = 183,
    Down,
    Right,
    Left,
    num,
    main
```

```
};

int main(int argc,const char* argv[])
{
}
```

注意：枚举是一种锦上添花的技术，使用它能让代码的可读性、安全性更高，但直接使用字面值数据也不影响代码的编写和运行。

二九、预处理与预处理指令

程序员所编译C代码不能被直接编译，它需要一段程序把它先翻译一下，被翻译过程预处理，负责翻译的程序叫预处理器，被翻译的指令叫预处理指令，C代码中以#开头的都是预处理指令。

```
gcc -E xxx.c 查看C代码的预处理结果，显示在终端
gcc -E xxx.c -o xxx.i 把预处理的结果保存到文件中，以.i结尾的文件也被称为预处理文件。
```

1、文件包含指令

#include 预处理指令的功能是导入一个头文件到当文件中，它用两中使用方法：

方法1：#include <file_name.h>

从系统指定的路径查找并导致头文件，一般用于导入标准库、系统、第三方库的头文件。

方法2：#include "file_name.h"

从系统当前路径查找并导致头文件，如果没有再从系统指定的路径查找并导致头文件，一般用于导入自定义头文件。

2、宏替换指令

1、定义宏名

#define 宏名 会被替换的内容

2、使用宏名

printf("%d\n",宏名);

注意：在预处理阶段，预处理器会把代码使用的宏名替换成宏名后面的那段内容。

3、宏常量

#define 宏名 字面值数据

给没有意义的字面值数据，取一个有意义名字，代替它，这样可以提高代码的可读性、可扩展性，还可以方便代码扩展。

```
#include <stdio.h>
#include <stdlib.h>

#define ARR_LEN 20


void show_arr(int arr[],size_t len)
{
    for(int i=0; i<len; i++)
    {
```

```
        printf("%d ",arr[i]);
    }
}

int main(int argc,const char* argv[])
{
    int arr[ARR_LEN];
    for(int i=0; i<ARR_LEN; i++)
    {
        arr[i] = rand() % 100;
    }

    show_arr(arr,ARR_LEN);
}
```

3、宏表达式

#define 宏名 表达式、操作、更复杂的标识符

```
#define STU_FORMAT "%s %c %hd %d"
#define sf scanf
#define YEAR_SEC (3600*24*365ul)
```

```
#include <stdio.h>

typedef struct Student
{
    int id;
    char name[20];
    char sex;
    float score;
}Student;

#define STU_FORMAT "%d %s %c %g"

int main(int argc,const char* argv[])
{
    Student stu;
    printf("请输入学生的学号、姓名、性别、成绩：");
    scanf(STU_FORMAT,&stu.id,stu.name,&stu.sex,&stu.score);
    printf(STU_FORMAT"\n",stu.id,stu.name,stu.sex,stu.score);
}
```

4、定义宏常量和宏表达式要注意的问题

由于宏常量和宏表达式可能使用在表达式中，因此在定义宏常量和宏表达式的末尾不要加分号。

5、枚举常量与宏常量的区别

提前：它们都可以提高代码的可读性，给没有意义字面值数据取一个有意义的名字。

1、从安全角度来说枚举常量要比宏常量安全，因为宏常量是通过替换实现的，在替换过程中可能会导致新的错误，如：有与宏名重名和函数或变量。

2、但枚举常量没有宏常量方便，枚举常量只能是整型数据，而宏常量可以是任何类型的，甚至是复杂的表达式，宏常量的使用范围更广。

总结：如果是大量的整型字面值数据建议定义为枚举常量，如果少量的或整型以外的类型的字面值数据建议定义为宏常量。

6、预定义的宏

编译器预定义的宏：

```
__FILE__ 获取当前文件名
__func__ 获取当前函数名
__LINE__ 获取当前行号
__DATE__ 获取当前日期
__TIME__ 获取当前时间
__WORDSIZE 获取当前编译器的位数
// 适合用来显示警告、错误信息。
```

```c
#include <stdio.h>

int main(int argc,const char* argv[])
{
    printf("%s\n",__FILE__);
    printf("%s\n",__func__);
    printf("%d\n",__LINE__);
    printf("%s\n",__DATE__);
    printf("%s\n",__TIME__);
}
```

标准库预定义的宏：

```c
// limits.h 头文件中定义的所有整数类型最大值、最小值
#define SCHAR_MIN (-128)
#define SCHAR_MAX 127
#define UCHAR_MAX 255

#define SHRT_MIN  (-32768)
#define SHRT_MAX  32767
#define USHRT_MAX 65535

#define INT_MIN  (-INT_MAX - 1)
#define INT_MAX  2147483647
#define UINT_MAX  4294967295U

#define LLONG_MAX   9223372036854775807LL
#define LLONG_MIN   (-LLONG_MAX - 1LL)
#define ULLONG_MAX  18446744073709551615ULL

// stdlib.h 头文件定义两个结标志
#define EXIT_SUCCESS (0)
#define EXIT_FAILURE (-1)

// stdbool.h 头文件定义了bool、true、false
#define bool    _Bool
#define true    1
#define false   0

// libio.h 头文件定义了NULL
#define NULL ((void*)0)
```

三十、宏函数

1、什么是宏函数

宏函数不是真正的函数，而是**带参数的宏替换**，只是使用方法像函数而已。

在代码中使用宏函数，预处理时会经历两次替换，第一次把宏函数替换成它后面的一串代码、表达式，第二次把宏函数中的参数替换到表达式中。

```
#define 宏名(a,b,c,...) a+b*c
```

```
#include <stdio.h>
#define ARR_SIZE(a) sizeof(a)/sizeof(a[0])

int main(int argc,const char* argv[])
{
    int arr[] = {1,2,3,4,4,3,2,1,5,5,6,6,8,7,9,9};
    // 第一次是把ARR_SIZE(arr)替换成了sizeof(a)/sizeof(a[0])，第二次替换是把a替换成了arr，最终ARR_SIZE(arr)变成了sizeof(arr)/sizeof(arr[0]);
    int len = ARR_SIZE(arr);
    for(int i=0; i<len; i++)
    {
        printf("%d ",arr[i]);
    }
}
```

2、定义宏函数要注意的问题

1、如果宏函数后面的代码有多行，可以使用大括号包括，进行保护。

```
#define 宏名(a,b,c,...) {代码1; 代码2; ...}
```

```
#include <stdio.h>
#define swap(a,b) {int t=a; a=b; b=t;}

int main(int argc,const char* argv[])
{
    int num1 = 3 , num2 = 7;
    swap(num1,num2);
    printf("%d %d\n",num1,num2);
}
```

2、宏函数后面的代码不能直接换行，如果代码确定太长，可以使用续行符换行。

```
#define 宏名(a,b,c,...) {  \
    代码1; \
    代码2; \
     ...   \
}
```

```
#include <stdio.h>

#define swap(a,b) { \
    int t=a;        \
```

```
    a=b;            \
    b=t;            \
}

int main(int argc,const char* argv[])
{
    int num1 = 3 , num2 = 7;
    swap(num1,num2);
    printf("%d %d\n",num1,num2);
}
```

3、使用了大括号保护的宏函数，在使用时末尾可以不加分号，为了统一语法的一致性，强制使用者在宏函数末尾加分号，可以使用do while语句来实现。

```
#define 宏名(a,b,c,...) do{  \
    代码1; \
    代码2; \
    ...   \
}while(0)
```

```
#include <stdio.h>

#define swap(a,b) do{ \
    int t=a;        \
    a=b;            \
    b=t;            \
}while(0)

int main(int argc,const char* argv[])
{
    int num1 = 3 , num2 = 7;
    swap(num1,num2);
    printf("%d %d\n",num1,num2);
}
```

4、为了防止宏函数出现二义性，对宏参数要尽量多加小括号。

3、宏的二义性

就是使用宏函数的环境不同、参数不同，造成宏函数有多执行规则，会出现出乎意料的执行结果，这种宏函数的二义性，设计宏函数时要尽量杜绝。

```
#include <stdio.h>

// #define SUM(a,b) a+b 这种写法就会造成二义性
#define SUM(a,b) (a+b)

int main(int argc,const char* argv[])
{
    printf("%d\n",SUM(5,4));

    int num = SUM(3,4)*12;
    printf("%d\n",num);
}

#include <stdio.h>
```

```c
// #define MUT(a,b) (a*b) 这种写法就会造成二义性
#define MUT(a,b) ((a)*(b))
int main(int argc,const char* argv[])
{
    printf("%d\n",MUT(3,4));
    printf("%d\n",MUT(3+2,4));
}

```

3、调用宏函数要注意的问题

1、传递给宏函数的参数不能使用自变运算符，因为我们无法知道参数在宏代码中会被替换多少次。

```c
#include <stdio.h>

#define MUT(a,b) ((a)*(a)*(b))

int main(int argc,const char* argv[])
{
    int num = 1;
    printf("%d\n",MUT(num++,4));
    printf("%d\n",num);
}
```

2、宏函数没有返回值，只是个别宏函数表达式有计算结果。

4、普通函数与宏函数的优缺点?

1、宏函数的优点

1、执行速度快，它不是真正的函数调用，而是代码替换，不会经历传参、跳转、返回值。

2、不会检查参数的类型，因此通用性强。

2、宏函数的缺点

1、由于它不是真正的函数调用，而是代码替换，每使用一次，就会替换出一份代码，会造成代码冗余、编译速度慢、可执行文件变大。

2、没有返回值，最多可以有个执行结果。

3、类型检查不严格，安全性低。

4、无法进行递归调用。

3、普通函数的优点

1、不存在代码冗余的情况，函数的代码只会在代码段中存储一份，使用时跳转过去执行，执行结束后再返回，还可以附加返回值。

2、安全性高，会对参数进行类型检查。

3、可以进行递归调用，实现分治算法。

4、函数的缺点

1、相比宏函数它的执行速度慢，调用时会经历传参、跳转、返回等过程，该过程耗费大量的时间。

2、类型专用，形参什么类型，实参必须是什么类型，无法通用。

5、什么样的代码适合封装成宏函数

1、代码量少，即使多次使用也不会造成代码段过度冗余。

2、调用次数少，但执行次数多，也就是宏函数会在循环语句中调用。

3、函数的功能对返回值没有要求，也就是函数的功能不是通过返回值达到的。

练习1、封装一个malloc、free函数。

my_malloc 它要记录my_malloc的调用位置(文件、函数、行号)，申请到的内存地址

my_free 它要记录my_free的调用位置(文件、函数、行号)，释放到的内存地址

```c
#include <stdio.h>
#include <stdlib.h>

void* _my_malloc(const char* file,const char* func,int line,size_t size)
{
    void* ptr = malloc(size);
    printf("%s %s %d malloc %p\n",__FILE__,__func__,__LINE__,ptr);
    return ptr;
}

#define my_malloc(size) _my_malloc(__FILE__,__func__,__LINE__,size)

#define my_free(ptr) do{ \
    printf("%s %s %d free %p\n",__FILE__,__func__,__LINE__,ptr); \
    free(ptr); \
}while(0)

int main(int argc,const char* argv[])
{
    int* ptr = my_malloc(40);
    *ptr = 123456;
    printf("%d\n",*ptr);
    my_free(ptr);
}
```

练习2、实现一个通用的变量交换函数。

```c
#include <stdio.h>
#include <string.h>

/*
只能适用于基本数据类型，结构、联合变量无法使用
#define swap(a,b) do{   \
    a = a+b;            \
    b = a-b;            \
    a = a-b;            \
}while(0)
*/

```

```
/*
使用时要多加一个参数
#define swap(a,b,type) do{  \
    type t = a;             \
    a = b;                  \
    b = t;                  \
}while(0)
*/

/*
typeof只能在gnu编译下使用
#define swap(a,b) do{   \
    typeof(a) t = a;    \
    a = b;              \
    b = t;              \
}while(0)
*/

#define swap(a,b) do{           \
    char tmp[sizeof(a)];        \
    memcpy(tmp,&a,sizeof(a));   \
    memcpy(&a,&b,sizeof(a));    \
    memcpy(&b,tmp,sizeof(a));   \
}while(0)

typedef struct Data
{
    char ch;
    int num;
}Data;

int main(int argc,const char* argv[])
{
    Data d1 = {1,1}, d2 = {2,2};
    printf("%d,%d %d,%d\n",d1.ch,d1.num,d2.ch,d2.num);
    swap(d1,d2);
    printf("%d,%d %d,%d\n",d1.ch,d1.num,d2.ch,d2.num);
}
```

三一、条件编译

条件语句(if、switch、for、while、do while)会根据条件选择执行哪些代码，条件编译就是预处理器根据条件选择哪些代码参与下一步的编译。

1、负责条件编译的预处理指令

```
#if
#else
#elif
#endif
#ifdef
#ifndef
```

2、头文件卫士

这种固定写法，在头文件中使用，它能防止头文件被重复包含，所有的头文件都要遵循这个规则。

```
#ifndef FILE_H // 判断FILE_H宏是否正在，不存在则条件为真
#define FILE_H // 定义FILE_H宏

// 头文件卫士能保证此处不重复出现

#endif//FILE_H // #ifndef的结尾
```

3、注释代码

```
// 只能注释单行代码，早期的编译器不支持该用

/* 多行注释，但不能嵌套 */

#if 0|1
可注释大块代码，可以嵌套
#endif
```

4、版本、环境判断

```
#if __WORDSIZE == 64
    typedef long int        int64_t;
#else
    typedef long long int       int64_t;
#endif


// 判断是否是Linux操作系统:
#if __linux__

#endif

// 判断是否是Windows操作系统:
#if __WIN32 | __WIN32__ | __WIN64__

#endif

int main(int argc,const char* argv[])
{
#if __linux__
    system("clear");
#elif ____WIN32 | __WIN32__ | __WIN64__
    system("cls");
#endif
    printf("hello world!\n");
}


// 判断gcc还是g++:
int main(int argc,const char* argv[])
{
#if __cplusplus
    printf("你使用是g++编译器\n");
```

```
#else
    printf("你使用是gcc编译器\n");
#endif
}

```

5、DEBUG宏

专门用于调试程序的宏函数，这种宏函数在程序测试、调试、试运行阶段执行，在程序正式上线阶段不执行，这类函数会根据DEBUG宏是否定义确定执行的流程。

```
一些操作提示，如：xxx操作成功，xxx操作失败，分配内存的记录、释放内存的记录，这类型消息开发人员、测试人员需要看到，但用户不需要看到。
```

```
#include <stdio.h>

#include <stdlib.h>

#ifdef DEBUG
void* _my_malloc(const char* file,const char* func,int line,size_t size)
{
    void* ptr = malloc(size);
    printf("%s %s %d malloc %p\n",__FILE__,__func__,__LINE__,ptr);
    return ptr;
}

#define my_malloc(size) _my_malloc(__FILE__,__func__,__LINE__,size)

#define my_free(ptr) do{ \
    printf("%s %s %d free %p\n",__FILE__,__func__,__LINE__,ptr); \
    free(ptr); \
}while(0)

#else

#define my_malloc malloc
#define my_free free

#endif

int main(int argc,const char* argv[])
{
    int* ptr = my_malloc(40);
    *ptr = 123456;
    printf("%d\n",*ptr);
    my_free(ptr);
}
```

三二、不常用的预处理指令

```
#line <常整数> 设置当前代码的行号，目前没有发现它有什么用

#error "在预处理阶段提示错误信息"，一旦预处理遇到它，将不再继续编译，它不能单独使用必须与条件判断系列语句配合使用

#warning "在预处理阶段提示警告信息" 不能建议单独使用，最好与条件判断系列语句配合使用。

#pragma GCC poison <标识符> 把标识符设置病毒，禁止在代码中使用

#pragma pack(n)  设置最大对齐和补齐字节数
  每个系统在进行对齐和补齐都有一个最大对齐和补齐字节数n，也就是超出n字节按n字节计算，例如：linux32系统n=4，windows32 n=8
设置要求：
    1、n < 系统默认的最大对齐、补齐字节数，往大了调整没有意义，速度不会提升还会导致内存浪费。
    2、n必须是2的x次方，也就是必须是1、2、4、8、16这一类的整数
```

宏函数的变长参数：

```
#define func(...) __VA_ARGS__
```

注意：这种用法必须配合，printf/fprintf/sprintf系列支持变长参数的函数使用。

在编译时定义宏：

```
gcc xxx.c -D ARR_LEN=3
-D ARR_LEN=3 <=> #define ARR_LEN 3 跟在代码中定义宏的效果一样

gcc xxx.c -D DEBUG
-D DEBUG <=> #define DEBUG
```

三三、多文件编程

当程序的业务逻辑越来越复杂，代码量越来越多，就需要多人组成团队协同开发，那么就必须把任务拆分成若干个文件。

1、一般的拆分方案

1、main.c 只当作程序的入口，不实现业务逻辑代码。

2、用于实现程序具体的业务逻辑代码

模块名.h 用说明.c文件中有哪些函数、全局变量，也就是函数声明、全局变量声明。

模块名.c 具体的函数实现，全局变量定义。

注意：不需要被其它.c使用的函数、全局变量，可以不在.h文件中声明，另外为了防止命名冲突、调用定义时可以用static修饰。

3、项目中常用的、通用的工具，宏函数、函数，一般取名为tools.h 、tools.c

4、只用于类型设计 type.h，里面包含有结构体、联合、枚举、宏常量、宏函数。

2、头文件中可以写什么

由于头文件可能会被若干个.c文件包含，那么每包含一次，.c文件中就会有一份头文件的内容，所以头文件中的内容必须可重复，因此我们只适合在头文件中实现以下内容：

1、头文件卫士

2、#include 语句

3、宏常量、宏函数

4、全局变量的声明(变量的声明可以有多份，但定义只能有一份)。

5、函数声明

6、结构、联合、枚举复合的类型设计

7、类型重定义

3、头文件包含可能出现的问题

> 头文件卫士只能解决重复包含的问题，但无法解决相互包含、递归包含的问题

互相包含:

a.h #include "b.h"

b.h #include "a.h"

递归包含：

a.h #include "b.h"

b.h #include "c.h"

c.h #include "a.h"

解决方法：

解决这种问题的文件，再设计一个.h文件，把他们共用的内容，实现在新的.h文件中，被他们共同包含即可。

4、多文件编译过程

1、gcc xxx.h 检查头文件是否有语法错误，如果没有语法错误，会生成xxx.h.gch，检查完毕后该文件要立即删除。

2、gcc -c xxx.c 把.c文件编译成二进制目标文件

3、gcc *.o 把所有目标文件合成可执行文件，也可以使用-o 设置可执行文件的名字。

三四、文件分类

1、二进制文件

把数据的补码直接写入文件，这种文件叫二进制文件。

优点：读写和写入时不需要进行转换，所以读写速度快，数据安全性高。

缺点：不能使用文本编译器打开，无法阅读。

2、文本文件

把数据转换成字符串写入文件，也就是把字符的二进制写入文件，这种文件叫文本文件。

优点：能被文本编辑器打开，人类能看的懂，能够看出数据是否出错。

缺点：读写时需要转换，读写速度慢，数据有被修改的风险。

3、二进制文件与文本文件的区别

> 假如有一个int类型的变量：int num = 12306;

二进制文件的内容：

00000000000000011000000010010

文本文件的内容：

先把num的值转换成字符串 “12306”，再把字符串中每个字符的ASCII的二进制写入到文件

49 50 51 48 54

0011000100110010001100110011000000110110

总结：二进制文件的大小是确定的，文本文件会根据数据的内容而变化，大小不确定。

三五、文件相关操作

1、打开与关闭的函数

```
FILE *fopen(const char *path, const char *mode);
功能：打开文件
path：文件的路径
mode：文件的打开模式
返回值：文件结构指针，是后续操作文件的凭证，失败则返回NULL。

int fclose(FILE *stream);
功能：关闭文件
返回值：成功返回0，失败返回-1。
注意：不能重复关闭，否则会出现double free错误，为了出现这种错误，在文件关闭及时把文件指针赋值为NULL，及时关闭文件可以把缓冲区中的数据写入到文件中。
```

2、文件的打开模式

```
"r" 以只读方式打开文本文件，如果文件不存在，或文件没有读权限则打开失败。
"r+" 在"r"的基础上增加了写权限。

"w" 以只写方式打开文本文件，如果文件不存在则创建，如果文件存在则清空文件的内容，如果文件存在但没有写权限，则打开失败。
"w+" 在"w"的基础上增加了读权限。

"a" 以只写方式打开文本文件，如果文件不存在则创建，如果文件存在则新写入的内容追加到文件末尾，如果文件存在但没有写权限，则打开失败。
"a+" 在"a"的基础上增加了读权限。

  如果要操作二进制文件，则在以模式的基础上增加b。
```

3、Windwos系统下以文本方式打开文件时

如果写入要的字符是'\n'时，Windows系统会写入'\n'和'\r'两个字符，在Windows系统中以'\n'+'\r'表示一个换行，也就是当写入数字0x0A时，会写入0x0A0D。

在读取数据时，遇到'\n''\r'只会读取一个'\n'，所以在Windows系统下，文件的打开模式b非常重要。

但在Linux、UNIX系统下加不加b则没有区别。

4、文本文件的读写

```
int fprintf(FILE *stream, const char *format, ...);
功能：把若干个变量以文本格式写入到指定的文件中
stream：要写入的文件
format：占位符+转义字符+提示信息
...：若干个变量
返回值：写入字符的数量

int fscanf(FILE *stream, const char *format, ...);
功能：从文件中读取数据
stream：要读取的文件
format：占位符
...：若干个变量的地址
返回值：成功读取的变量个数
```

```
#include <stdio.h>

typedef struct Data
{
    char ch;
    short sh;
    int num;
    float sf;
}Data;

int main(int argc,const char* argv[])
{
    FILE* wfp = fopen("test.txt","w");
    if(NULL == wfp)
    {
        printf("打开文件失败！");
        return 0;
    }

    Data d = {65,32766,123456789,3.14};
    int ret = fprintf(wfp,"%c %hd %d %f\n",d.ch,d.sh,d.num,d.sf);
    printf("写入了%d个字符\n",ret);

    fclose(wfp);
}

int main(int argc,const char* argv[])
{
    FILE* rfp = fopen("test.txt","r");
    if(NULL == rfp)
    {
        printf("打开文件失败！");
```

```
        return 0;
    }

    Data* p = malloc(sizeof(Data)) ;
    int ret = fscanf(rfp,"%c %hd %d %f",&p->ch,&p->sh,&p->num,&p->sf);
    printf("读取了%d个变量\n",ret);
    printf("%c %hd %d %f",p->ch,p->sh,p->num,p->sf);

    fclose(rfp);
}
```

练习：设计一个图书结构体(书名、版号、作者、出版社)，定义一个图书结构变量并初始化，使用fprintf写入一个图书结构变量的数据到book.txt文件，从book.txt文件中读取一个条数据到图书结构变量中。

```
void init_cts(void)
{
    // 如申请内存并初始化
    cts = malloc(sizeof(Contacts)*CTS_MAX);
    bzero(cts,sizeof(Contacts)*CTS_MAX);

    // 打开文件
    FILE* rfp = fopen("cts.txt","r");
    if(NULL == rfp)
    {
#ifdef DEBUG
        printf("文件打开失败，如果是第一次运行请忽略，否则请检查数据文件的权限！\n");
#endif
        return;
    }

    while(3 == fscanf(rfp,"%s %c %s\n",cts[cnt].name,&cts[cnt].sex,cts[cnt].phone))
    {
        cnt++;
    }

#ifdef DEBUG
    printf("成功加载%d条数据！\n",cnt);
#endif

    fclose(rfp);
}

void exit_cts(void)
{
    FILE* wfp = fopen("cts.txt","w");
    if(NULL == wfp)
    {
#ifdef DEBUG
        printf("创建文件失败，请检查当前路径的权限或查看磁盘的使用情况！\n");
#endif
        free(cts);
        return;
    }

    for(int i=0; i<cnt; i++)
    {
```

```
        fprintf(wfp,"%s %c %s\n",cts[i].name,cts[i].sex,cts[i].phone);
    }

#ifdef DEBUG
    printf("保存了%d条数据！\n",cnt);
#endif

    fclose(wfp);
    free(cts);
}
```

5、二进制文件的读写

```
size_t fwrite(const void *ptr, size_t size, size_t nmemb,FILE *stream);
功能：把一块内存当作数组，然后数组中的内容以二进制格式写入到文件中
ptr：数组首地址
size：数组元素的字节数
nmemb：数组的长度
stream：要写入的文件
返回值：实际写入的次数

size_t fread(void *ptr, size_t size, size_t nmemb, FILE *stream);
功能：把二进制文件中的内容读取的数组中
ptr：要存储数据的数组首地址
size：数组元素的字节数
nmemb：数组的容量
返回值：成功读取的次数
注意：以二进制格式读写文件时，最好加上mode最好包含b。
```

```
#include <stdio.h>
#include <stdlib.h>
#define ARR_MAX 15

// 写入二进制数据
int main(int argc,const char* argv[])
{
    int arr[ARR_MAX];
    for(int i=0; i<ARR_MAX; i++)
    {
        arr[i] = rand() % 100;
    }

    FILE* wfp = fopen("arr.bin","wb");
    if(NULL == wfp)
    {

        printf("创建文件失败，请检查路径权限！\n");
        return 0;
    }

    int ret = fwrite(arr,sizeof(arr[0]),ARR_MAX,wfp);
    printf("保存了%d个数组元素！\n",ret);

    fclose(wfp);
}

```

```c
#include <stdio.h>
#include <stdlib.h>
#define ARR_MAX 15
// 读取二进制数据
int main(int argc,const char* argv[])
{
    FILE* rfp = fopen("arr.bin","rb");
    if(NULL == rfp)
    {
        printf("打开文件失败！\n");
        return 0;
    }

    int* p = malloc(sizeof(p[0])*ARR_MAX);
    int ret = fread(p,sizeof(p[0]),ARR_MAX,rfp);
    printf("读取%d个数组元素！\n",ret);

    for(int i=0; i<ret; i++)

    {
        printf("%d ",p[i]);
    }

    fclose(rfp);
}
```

注意：如果以fwrite/fread读写的字符数组，那么我们操作的依然是文本文件。

```c
#include <stdio.h>

#include <stdlib.h>
#define ARR_MAX 15

int main(int argc,const char* argv[])
{
    char arr[ARR_MAX];
    for(int i=0; i<ARR_MAX; i++)
    {
        arr[i] = rand() % 26 + 65;
    }

    FILE* wfp = fopen("arr.bin","w");
    if(NULL == wfp)
    {
        printf("打开文件失败！\n");
        return 0;
    }

    fwrite(arr,1,ARR_MAX,wfp);
    fclose(wfp);
}
```

```
int sprintf(char *str, const char *format, ...);
功能：把若干个变量转换成字符串输出到str数组中

int sscanf(const char *str, const char *format, ...);
功能：从字符串读取若干个变量
```

```c
#include <stdio.h>
#include <string.h>

typedef struct Data
{
    char ch;
    short sh;
    int num;
    float sf;
}Data;

int main(int argc,const char* argv[])
{
    Data d = {97,3233,987654321,3.14};
    char buf[255];

    // 把结构体转换成字符串后，可以网络发送，数据库存储，文件存储
    sprintf(buf,"%c %hd %d %f\n",d.ch,d.sh,d.num,d.sf);
    printf("%s\n",buf);

    FILE* wfp = fopen("data.bin","w");
    if(NULL == wfp)
    {
        printf("打开文件失败！\n");
        return 0;
    }

    fwrite(buf,1,strlen(buf),wfp);
    fclose(wfp);
}

#include <stdio.h>
typedef struct Data
{
    char ch;
    short sh;
    int num;
    float sf;
}Data;

int main(int argc,const char* argv[])
{
    FILE* rfp = fopen("data.bin","r");
    if(NULL == rfp)
    {
        printf("打开文件失败！\n");
        return 0;
    }

    char buf[255];
```

```c
    fgets(buf,sizeof(buf),rfp);
    fclose(rfp);

    // 网通发送，数据库存储，转换成结构变量
    Data d;
    sscanf(buf,"%c %hd %d %f\n",&d.ch,&d.sh,&d.num,&d.sf);
    printf("--%c %hd %d %f--\n",d.ch,d.sh,d.num,d.sf);
}
```

6、文件位置指针：

文件位置指针它记录着读写文件时的位置，读取数据时会从文件位置指针指向的位置开始读取，写入数据时也会写入到文件位置指针所指向的位置，并且它会随着读写操作自动移动。

```
以"r"、"r+"方式打开文件，文件位置指针指向文件的开头，以"a"、"a+"方式打开文件，文件位置指针指向文件的末尾。
fprintf/fwrite写入数据后立即读取，之所以会失败，是因为文件位置指针指向着文件的末尾。
```

```c
void rewind(FILE *stream);
功能：把文件的位置指针调整到文件的开头。

long ftell(FILE *stream);
功能：返回文件位置指针指向了文件中的第几个字节

int fseek(FILE *stream, long offset, int whence);
功能：设置文件的位置指针
stream：要设置的文件
offset：偏移值(正负整数)
whence：基础位置
    SEEK_SET 文件开头
    SEEK_CUR 当前位置
    SEEK_END 文件末尾
whence+offset就是文件指针最终设置的位置。
返回值：成功返回0，失败返回-1。
```

```c
#include <stdlib.h>
#define ARR_MAX 15

int main(int argc,const char* argv[])
{
    int arr[ARR_MAX];
    for(int i=0; i<ARR_MAX; i++)
    {
        arr[i] = rand() % 100;
    }

    FILE* rwfp = fopen("arr.bin","w+");
    if(NULL == rwfp)
    {
        printf("打开文件失败！\n");
        return 0;
    }

    int ret = fwrite(arr,1,ARR_MAX*sizeof(arr[0]),rwfp);
```

```
    printf("写入了%d个字节，此时文件位置指针指向了文件的%ld个字节\n",ret,ftell(rwfp));

    // 把文件的位置指针设置到文件的开头
    //rewind(rwfp);
    //fseek(rwfp,0,SEEK_SET);
    //fseek(rwfp,-60,SEEK_END);
    fseek(rwfp,-60,SEEK_CUR);

    int* p = malloc(sizeof(p[0])*ARR_MAX);
    ret = fread(p,1,ARR_MAX*sizeof(p[0]),rwfp);
    printf("读取%d个字节\n",ret);

    for(int i=0; i<ARR_MAX; i++)
    {
        printf("%d ",p[i]);
    }

    fclose(rwfp);
}
```

7、文件操作时的局限性：

文件的内容是连续存储在磁盘上的，所以就导致以下操作：

向文件中插入数据：

1、文件位置指针调整到要插入的位置。

2、把后续的数据整体向后拷贝n(要插入的数据字节数)个字节。

3、文件位置指针调整到要插入的位置，写入数据。

从文件中删除数据：

1、文件位置指针调整到要删除的数据末尾。

2、把后续的数据整体向前拷贝nn(要删除的数据字节数)个字节。

3、修改文件的大小。

```
**注意：**所以，程序运行时，先把数据从文件中加载内存，程序在运行期间只针对内存进行增、删、改、查，程序结束时再把数据从内存写入文件。
```

```
#include <stdio.h>

int main(int argc,const char* argv[])
{
    // 已经test.txt文件中存储的是ABCDEFG，我们要在EF中间插入hehe
    FILE* rwfp = fopen("test.txt","r+");
    if(NULL == rwfp)
    {
        printf("打开文件失败！\n");
        return 0;
    }

    char* str = "hehe";

```

```c

    // 把文件位置指针调整到E的后面/F的前面
    fseek(rwfp,5,SEEK_SET);

    char buf[4096];
    int ret = fread(buf,1,sizeof(buf),rwfp);

    fseek(rwfp,5+strlen(str),SEEK_SET);
    fwrite(buf,1,ret,rwfp);

    fseek(rwfp,5,SEEK_SET);
    fwrite(str,1,strlen(str),rwfp);


    fclose(rwfp);
}
```

8、文件管理

```c
int remove(const char *pathname);
功能：删除文件

int rename(const char *oldpath, const char *newpath);
功能：重命名文件

int truncate(const char *path, off_t length);
功能：把文件的内容设置为length字节数

char *tmpnam(char *name);
功能：生成一个与当前文件系统不重名的文件名。

int access(const char *pathname, int mode);
功能：检查文件的权限
mode:
    R_OK 读权限
    W_OK 写权限
    X_OK 执行权限
    F_OK 文件是否存在
返回值：
    检查的权限如果存在则返回0，不存在则返回-1。
```